

Manual de conectividad

Integración con SOAP



© 2013 Meta4 Spain, S.A. Se reservan todos los derechos.

AVISO: Este manual está protegido por la legislación referente a propiedad intelectual e industrial y por tratados internacionales. La utilización permitida de esta documentación queda limitada a su uso en conexión con el producto, y todo uso no autorizado será perseguido de acuerdo con la legislación aplicable. Se prohíbe su copia, modificación, reproducción o distribución sin permiso del titular.

Meta4 PeopleNet © 1999 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 KnowNet © 1996 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 e-mind © 2001 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 PeopleNet Ksystem © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4 t.innova © 2003 Meta4 Spain, S.A. Se reservan todos los derechos.

Meta4[®], Meta4Mind[®], Meta4 PeopleNet[®], Meta4 KnowNet[®], Meta4 e-mind[®], Meta4 PeopleNet Ksystem[®] y Meta4 t.innova[®] son marcas registradas propiedad de Meta4Spain, S.A.

Otros nombres de compañías, productos o servicios son marcas registradas o nombres comerciales de sus respectivos propietarios.

Meta4 Spain, S.A.
Centro Europa Empresarial
Edificio Roma
C/ Rozabella, 8
Ctra. de La Coruña, km 24,200
28290 Las Rozas, Madrid
SPAIN
<http://www.meta4.com>

Fecha de creación: enero de 2003.

Fecha de la última publicación: diciembre de 2013.

Tabla de contenidos

Introducción

■ Acerca de este manual	7
Propósito	7
Audiencia	7
Requisitos	8

1 SOAP en la arquitectura de las aplicaciones Meta4

■ Acerca de este capítulo	9
■ ¿Qué es SOAP?	10
Definición de SOAP	10
Uso de SOAP	10
WSDL para los mensajes SOAP	11
■ Arquitectura de servicios SOAP	12
■ Integración de SOAP en la arquitectura de Meta4	14
Exportar los Meta4Objects como servicios SOAP	14
Crear un Meta4Object cliente SOAP	15
■ Características de Meta4 como servicio SOAP	16
Características generales de los métodos de negocio de Meta4	16
Características avanzadas de los métodos de negocio de Meta4	18
Operaciones de persistencia	18
Autenticación	19
Ficheros	19
Estilo/Uso	20
Compatibilidad	20
■ Características de Meta4 como cliente SOAP	21
Servicios SOAP que pueden consumirse desde Meta4	21

2 Meta4 como servicio SOAP: aspectos prácticos

■ Acerca de este capítulo	23
■ Instalación de componentes necesarias	24

Recomendaciones de seguridad para la instalación de las aplicaciones de Meta4	26
Usuarios de arranque	26
■ Creación y publicación de los métodos de negocio	27
Creación del objeto de negocio con sus métodos	27
Dar de alta los métodos de negocio.	28
■ Procedimiento de generación y activación de servicios.	30
Procedimiento manual desde las páginas de administración	30
Configuración del despliegue de los servicios	34
Configuración de server-config.wsdd	34
Configuración del filtro SOAP	35
Procedimiento automático con identificador de servicio	35
Llamar al servicio exportado	36

3 Meta4 como cliente SOAP: aspectos prácticos

■ Acerca de este capítulo	39
■ Invocar servicios SOAP a través de Java	40
Configuración de la máquina virtual Java	40
Creación de un cliente SOAP	41
Creación de un Meta4Object para llamar a las clases correspondientes	42

4 Los servicios SOAP de Meta4 en detalle

■ Acerca de este capítulo	45
Ciclo completo de desarrollo y ejecución de un servicio SOAP con operaciones de consulta y persistencia	45
Crear objeto y métodos de negocio	46
Publicar los métodos de negocio	47
Obtener descripción del servicio	49
Crear las clases proxy	49
Elaborar el código cliente	50
Realizar verificación del servicio	55
Correspondencia del identificador con una sesión	55
Reportar errores de persistencia como excepciones SOAP	55
Borrado de jerarquías	56
Servicio sobre un Meta4Object con soporte a ficheros	57
Autenticación de los servicios SOAP.	60
Autenticación clásica (ejecución de servicio de login)	60
Codificación	61
Autenticación básica HTTP	63
Codificación	66

Autenticación basada en WS-Security	66
Codificación	68
Autenticación integrada con seguridad J2EE	70

5 Los clientes SOAP de Meta4 en detalle

■ Acerca de este capítulo	73
Cliente de un servicio SOAP de autenticación clásica de Meta4 (ejecución del servicio de login)	73
Cliente de un servicio SOAP de Meta4 (con operaciones de consulta)	75
Cliente de un servicio SOAP de Meta4 con un fichero de entrada	81

Introducción

Acerca de este manual

Propósito

En este manual se presenta la posibilidad que ofrece la tecnología de Meta4 de integrar sus productos como Meta4 PeopleNet, Meta4 KnowNet, Meta4 e-mind o Meta4 t.innova con otras aplicaciones o servicios externos, por medio de servicios SOAP (*Simple Object Access Protocol*).

El objetivo que se pretende alcanzar con este manual consiste en describir los conceptos, la funcionalidad y la configuración de los servicios SOAP, y parametrizar la funcionalidad Meta4 utilizando los servicios SOAP de la tecnología Meta4.

El manual se estructura de la siguiente manera:

- Introducción
- SOAP en la arquitectura de las aplicaciones Meta4
- Meta4 como un servicio SOAP
- Meta4 como un cliente SOAP

Audiencia

Este manual está dirigido a administradores del sistema, personas involucradas en la implantación y desarrollo de proyectos de integración con aplicaciones o servicios externos.

Requisitos

Se asume que el lector tiene conocimientos de:

- XML y servicios Web.
- la arquitectura de la aplicación Meta4 y la estructura de sus componentes.
- la estructura de los Meta4Objects y componentes relacionados.

SOAP en la arquitectura de las aplicaciones Meta4

1

Acerca de este capítulo

SOAP es un protocolo ligero, simple basado en XML, cuyo propósito es intercambiar información estructurada y clasificada en objetos en el entorno Web.

La característica más notable de SOAP es que se ha implantado en diferentes plataformas de hardware y software, lo que significa que puede utilizarse para enlazar sistemas dispares dentro y fuera de su empresa.

Este capítulo explica los conceptos y los procesos básicos de SOAP, cómo se integra en la arquitectura Meta4, y cómo se configuran y activan los servicios SOAP en caso de querer utilizarlos.

¿Qué es SOAP?

Definición de SOAP

SOAP (*Simple Object Access Protocol*) es un estándar de protocolo para el intercambio de información distribuida. Este protocolo define los mensajes XML que deben codificarse para construir la petición y respuesta entre el emisor y el receptor. Si se escribe un cierto fragmento de XML y se incluye dentro de una envoltura SOAP, esto constituye un mensaje SOAP.

EJEMPLO

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope
  xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Body>
    <ns1:test
      soapenv:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
      xmlns:ns1="http://soapinterop.org/">
      <arg0 xsi:type="xsd:dateTime">2002-12-13T11:34:54.000Z</arg0>
      <arg1 xsi:type="xsd:double">-34.0</arg0>
      <arg2 xsi:type="xsd:string">this is a test</arg0>
    </ns1:test>
  </soapenv:Body>
</soapenv:Envelope>
```

Normalmente se utiliza el protocolo HTTP para el transporte de los mensajes entre el cliente y el servidor, aunque es posible utilizar otros protocolos como SMTP para transportar dichos mensajes.

Uso de SOAP

Para utilizar SOAP, no se desarrollan ni escriben mensajes SOAP directamente, sino se utiliza un Toolkit de SOAP para ello. Generalmente, estos kits de herramientas traducen llamadas de función de algún tipo de lenguaje a un mensaje SOAP. Por ejemplo, Microsoft SOAP Toolkit 2.0 traduce llamadas de función COM a SOAP y el kit de herramientas de Apache AXIS traduce funciones de llamadas Java a SOAP.

WSDL para los mensajes SOAP

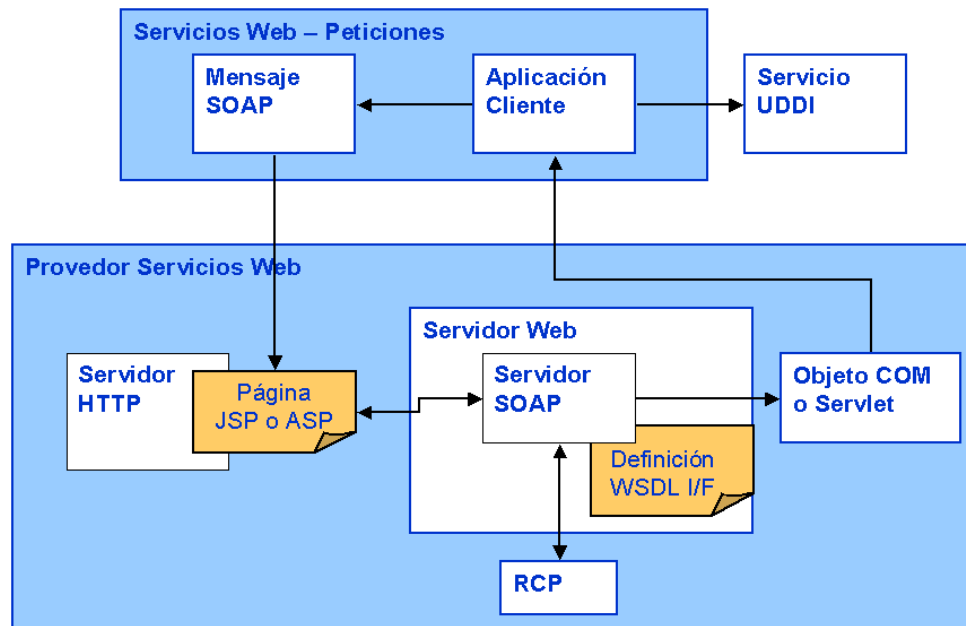
Cuando un servidor exporta un servicio web XML, debe informar a los clientes del modo en que pueden llamar a dicho servicio. Para ello, exporta además un archivo en formato WSDL (*Web Services Description Language*), que describe las características del servicio. Se puede definir un archivo WSDL como un documento XML que describe los posibles mensajes SOAP que pueden enviarse a un servicio y la forma en la que éstos se intercambian. WSDL es a SOAP lo que IDL es a CORBA o a COM.

Además de describir el contenido de un mensaje, WSDL define el lugar en el que está disponible el servicio y qué protocolo de comunicaciones se utiliza para hablar con el servicio. Existen varias herramientas disponibles para leer un archivo WSDL y generar el código necesario para establecer comunicaciones con un servicio Web XML.

Arquitectura de servicios SOAP

Como se ha descrito en el apartado anterior SOAP especifica el formato de los mensajes XML que pueden intercambiar un cliente y un servidor para realizar llamadas a métodos remotos (RPC). Si un cliente quiere realizar llamadas SOAP a un servidor, el servidor exportará un servicio con uno o varios métodos, y el cliente, que puede haber sido construido en otro sistema operativo, tecnología o lenguaje de programación, llamará a los métodos que el servicio contiene. Una gran ventaja de las llamadas a métodos remotos a través del protocolo SOAP es que pueden hacerse a través de HTTP, que es un protocolo ubicuo y que los *firewalls* y *proxies* no suelen bloquear.

Figura 1. Arquitectura típica que emplea servicios SOAP



En detalle, los mensajes SOAP fluirían del siguiente modo:

1. La aplicación cliente SOAP configura la dirección del servicio (incluyendo el servidor y el puerto).
2. La aplicación cliente SOAP construye un mensaje de petición SOAP, que es un documento XML que expresa lo que se quiere solicitar del servidor SOAP.

Para construir el mensaje correctamente, es posible que el desarrollador de la aplicación cliente SOAP tenga conocimiento de la descripción del servicio (WSDL) o que utilice una herramienta capaz de generar el código cliente necesario para que, a partir de la descripción del servicio, se pudiera construir automáticamente el código necesario (como WSDL2Java).

-
3. La aplicación cliente SOAP envía el mensaje SOAP al servidor SOAP. En el servidor, un servlet/objeto COM o página JSP/ASP escucha la petición.
 4. El servidor SOAP parsea el paquete SOAP e invoca el método adecuado del objeto al que está asociado el servicio, pasándole los parámetros incluidos en el documento SOAP. Opcionalmente, las cabeceras SOAP pueden dar lugar a un procesamiento previo.
 5. El objeto al que está asociado el servicio hace lo que se le ha encargado y devuelve datos al servidor SOAP, que empaqueta la respuesta en un sobre SOAP.
 6. El cliente recibe la respuesta, desempaqueta el sobre SOAP y envía la respuesta a la aplicación cliente que lo solicitó, completando de este modo el ciclo de petición y respuesta.

Integración de SOAP en la arquitectura de Meta4

Para mejorar la flexibilidad y alcance de su integración con otras aplicaciones, Meta4 ha decidido hacer posible que sus productos puedan ser servidor SOAP (los métodos de un Meta4Object puedan ser llamados por un cliente SOAP cualquiera) y cliente SOAP (un Meta4Object pueda realizar llamadas a los métodos de un servicio SOAP externo cualquiera).

Al facilitar la integración con SOAP, las posibilidades de Meta4 de conectividad con aplicaciones y servicios externos se completan del siguiente modo:

Requisito	Solución antes de SOAP	Solución con SOAP
Otras aplicaciones deben llamar a la aplicación de Meta4	RPC entre objetos via protocolos binarios (DCOM para exportar los Meta4Objects) o uso a bajo nivel de M4XML.	Exportar los Meta4Objects como servicios SOAP (esta solución presenta tantas ventajas que las anteriores soluciones entrantes ya no están aconsejadas)
La aplicación de Meta4 debe llamar a otras aplicaciones	Hacer una librería de conector genérico (se necesita librería cliente de terceros del sistema externo).	Crear un Meta4Objects cliente SOAP (se necesita que la aplicación externa exporte servicios SOAP).

Exportar los Meta4Objects como servicios SOAP

La plataforma Meta4 puede funcionar como servidor SOAP, puesto que incluye un motor de SOAP (Apache AXIS) y todas las utilidades necesarias para que ciertos métodos de un Meta4Object puedan ser exportados como métodos de un servicio SOAP.

Los servicios SOAP de Meta4 están escritos en Java. Tienen la particularidad de que se generan de modo asistido. Es decir, que para exportar un método de un Meta4Object mediante SOAP no se necesita desarrollar manualmente código Java, sino que basta con utilizar herramientas que se proporcionan para que su código se pueda generar, compilar y distribuir a partir del repositorio de base de datos de Meta4.

La arquitectura en la que se ejecutan los servicios SOAP generados por Meta4 se consta de los siguientes componentes:

- Contenedor Web: Tomcat o JRun según la *Guía de Certificaciones*.
- Motor de SOAP: Apache AXIS
- Aplicación Web: Aplicación Web de Meta4 (servicios Web o «cliente ligero»).

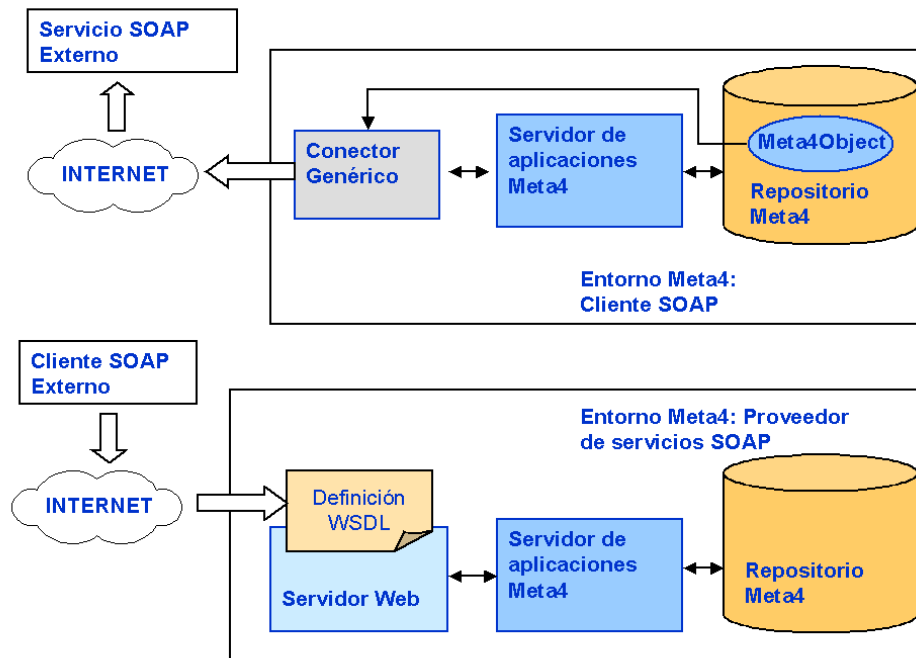
- Servicios generados automáticamente: Su código se genera, compila y ejecuta dentro de la aplicación Web de Meta4.

Meta4 proporciona el contenedor Web, el motor de SOAP y la aplicación Web, y distribuye utilidades para que se puedan generar, compilar y ejecutar los servicios.

Para llamar a los servicios generados automáticamente, se puede usar cualquier cliente externo de SOAP, desarrollado en cualquier lenguaje.

Por ejemplo si quiere utilizar un cliente desarrollado en Java, el archivo WSDL de descripción del servicio, que se publica automáticamente junto con el servicio SOAP para poder crear fácilmente las clases cliente desde las que invocar al servicio desde un entorno de desarrollo como Eclipse o similar

Figura 2. Meta4 como cliente SOAP y proveedor de servicios SOAP



Crear un Meta4Object cliente SOAP

La aplicación Meta4 también puede funcionar como un cliente SOAP que realiza peticiones a un determinado servicio SOAP externo y traspasa los datos requeridos a un Meta4Object.

Para que los Meta4Objects puedan realizar llamadas a un servicio SOAP de cuyo WSDL se disponga, la tecnología base de las aplicaciones de Meta4 utiliza:

- La infraestructura de invocación a Java desde el server: Unas clases C++ que junto con unas librerías de Java, permiten realizar llamadas a Java desde el cliente de desarrollo o el servidor de aplicaciones.
- Las utilidades de ayuda para gestionar los ficheros que se generan cuando a partir de una descripción de servicio (WSDL) se crean unas clases cliente para poder invocarlo: una presentación y unos Meta4Objects que guardan en tablas de repositorio los ficheros JAR generados como cliente de los servicios SOAP.

Características de Meta4 como servicio SOAP

Como hemos visto, un servicio SOAP puede estar formado por uno o varios métodos, con sus argumentos y su valor de retorno, a los que los clientes SOAP envían mensajes de entrada, esperando mensajes de salida.

Los métodos de un servicio SOAP suelen encapsular funcionalidad de la capa de negocio, por lo cual reciben el nombre de métodos de negocio. Los objetos que los contienen se conocen como objetos de negocio.

Según el tipo de argumentos que acepta un método de negocio, puede ser:

- **Método básico:** Son los que tienen solamente argumentos simples. Los argumentos simples pueden ser de tipo cadena (un número de caracteres de longitud finita), número (valores de coma flotante de 64 bits) o fecha (instantes específicos de tiempo).
- **Método complejo:** Son los que tienen argumentos complejos, o una mezcla de argumentos simples y complejos. Un argumento complejo es, por ejemplo, una estructura de tipos básicos, un vector de tipos básicos, un vector de estructuras de tipos básicos o una estructura de estructuras de tipos básicos.

Características generales de los métodos de negocio de Meta4

En la arquitectura SOA de Meta4, el Meta4Object se corresponde con el objeto de negocio y los argumentos básicos se corresponden con los argumentos del método (o concepto) que representa el método de negocio. Solamente existen argumentos básicos de entrada, no de salida.

Los argumentos complejos, también conocidos como argumentos de tipo nodo, se corresponden con nodos del Meta4Object donde se define el método de negocio. Estos argumentos son de salida, si el nodo se devuelve al llamar al método, y de entrada, si el nodo se rellena al realizar la llamada.

EJEMPLO

Supongamos un Meta4Object ZZ_TK_PERSON_SOAP multinodo, con un nodo sobre la tabla de personas (STD_PERSON), y un nodo hijo sobre la tabla de correos electrónicos (STD_EMAIL).

El objeto de negocio es ZZ_TK_PERSON_SOAP, por tanto el servicio tendría el mismo identificador, ZZ_TK_PERSON_SOAP.

Se definirán tantos métodos de negocio como funcionalidades se deseen exportar.

- Un ejemplo de método de negocio básico sería getAge, que dado un identificador de empleado, recuperara su edad en función de su fecha de nacimiento.
- Un método de negocio complejo de salida sería getPerson, que dado un identificador de empleado, recuperara todas sus direcciones postales y electrónicas
- Un método de negocio complejo de entrada sería putPerson, que dado un identificador de empleado, modificara información relativa a la misma, como su estado civil.

Para que un método LN4 pueda utilizarse por SOAP, debe publicarse, es decir, debe darse de alta en la tabla que declara los métodos públicos exportados por SOAP.

Por cada Meta4Object solamente puede declararse un servicio, cuyo nombre es el mismo que el del Meta4Object.

Para elegir los métodos (o conceptos) que serán publicados como métodos de negocio, se puede utilizar una presentación que se inicia desde el diseñador de Meta4Objects. Al publicar, se puede elegir el nombre público del método de negocio y hacerlo distinto al del método (o concepto) LN4. Existe una correspondencia 1:n entre métodos (o conceptos) LN4 y métodos de negocio, es decir, por cada método (o concepto) LN4 de un Meta4Object se pueden definir varios métodos de negocio, con distintos nombres, con o sin argumentos complejos, o siendo estos de entrada o de salida.

A la hora de definir un nodo como argumento complejo, es necesario que dicho nodo sea raíz, para evitar que existan problemas de posicionamiento.

Cuando se exporta un nodo en un argumento de salida, se exporta el nodo y todos sus hijos. Para ello, en el código del método se deben utilizar métodos de carga (como la ROOTLOAD) que consigan cargar el nodo raíz y sus hijos.

Cuando se rellena un nodo en un argumento de entrada, para elementos de tipo registro se realiza una adición de registro si no se especifica un identificador de registro (RID), y una modificación si se identifica. Para realizar un borrado de registro, es preciso marcar el registro para borrado. También está previsto que los métodos persistan los nodos raíz y todos sus hijos, para lo

que es preciso utilizar métodos de persistencia que lo cumplan (como PERSIST_TREE).

En cuanto a los valores de retorno, mientras que el método LN4 retorna tipos simples (cadenas, números o fechas), los métodos de negocio de Meta4 retornan un tipo complejo que contiene, además del valor de retorno del método LN4, un vector de mensajes de traza (errores o avisos).

Existe una forma rápida de publicar ciertos servicios SOAP desde la herramienta de la Consulta, cuando solamente se desea realizar una consulta de datos. Si se indica que una consulta está exportada por SOAP, la herramienta de la consulta crea un método predeterminado para obtener dichos datos, y exporta el nodo raíz de la misma como argumento de salida de un método complejo, de forma transparente al usuario.

Características avanzadas de los métodos de negocio de Meta4

Operaciones de persistencia

Como se ha indicado anteriormente, los métodos complejos con argumentos de entrada no distinguen en su lógica de negocio si el desarrollador desea añadir, modificar o borrar elementos, sino que se apoyan en elementos auxiliares para diferenciar dichas funciones. El identificador de registro (RID) es un número entero que designa de modo único un registro cargado en una instancia de máquina virtual dentro del servidor de aplicaciones. Para obtener dicho valor, es necesario realizar una transacción que levante dicha instancia. La marca de borrado, es un booleano que indica si el registro identificado por el RID debe ser eliminado.

:

Operación de persistencia	Identificador de registro (RID)	Marca de borrado
Añadir registros: Construir un argumento nodo de entrada con la información necesaria, ejecutar el método de negocio.	No	No

Operación de persistencia	Identificador de registro (RID)	Marca de borrado
Modificar registros: Realizar una operación de consulta sobre el nodo que contiene el registro en el que se realizará la modificación, obteniendo el identificador tecnológico del mismo. Una vez obtenido dicho identificador, construir un argumento nodo de entrada que contenga la información a modificar, así como dicho identificador tecnológico, e incluirlos en la llamada al método de negocio.	Sí	No
Borrar registros: Igual que en la modificación de registros, se debe realizar una operación de consulta sobre el nodo que contiene el registro en el que se realizará la modificación, obteniendo el identificador tecnológico del mismo. A continuación, construir un argumento nodo de entrada que contenga el identificador del registro a borrar , y la marca para borrado del mismo, e incluirlos en la llamada al método de negocio.	Sí	Sí

Autenticación

Entre los conceptos aplicables a la seguridad de los servicios SOAP de Meta4 se encuentra el concepto de sesión para SOAP. Para acceder a un servicio SOAP de Meta4 se debe disponer de una sesión válida en el servidor de aplicaciones Meta4 asociada al contexto HTTP del transporte entre cliente y servidor. No es posible acceder a un servicio SOAP de Meta4 sin estar autenticado, lo cual garantiza la confidencialidad de los datos exportados desde la plataforma.

Una sesión para SOAP en Meta4 puede ser obtenida por diversos métodos, entre los cuales se halla:

- La ejecución de un método de un servicio previo, llamado servicio de login de Meta4, que crea una sesión y devuelve su identificador.
- La autenticación básica del protocolo HTTP.
- La implementación de Username Token Profile 1.0 (estándar de seguridad de WS-Security)
- Mediante SSO con contenedores web J2EE

Ficheros

Los servicios SOAP de Meta4 soportan el envío y la recepción de ficheros, pero

estos viajan dentro de argumentos complejos de salida o de entrada, no dentro de los argumentos básicos o de los tipos de retorno.

Meta4 utiliza el estándar SwA (SOAP with Attachments), para intercambiar ficheros. Los ficheros son transportados en adjuntos MIME, al margen del mensaje SOAP, de forma parecida a cómo se añaden los ficheros adjuntos a un mensaje de correo electrónico.

El estándar SwA no resuelve la interoperabilidad del WSDL, y los ficheros de los servicios SOAP de Meta4 aparecen reflejados como un tipo complejo llamado `apachesoap:DataHandler`, por lo que se requiere un cliente con conocimiento de este tipo de datos (Java) para poder consumirlos, o un servicio puente a una implementación que el cliente consuma.

EJEMPLO

Sea un `Meta4Object` `TST_SOAP_XMLFILE` cuyo nodo raíz está vinculado a una tabla que contiene ficheros.

- Se podría definir un método de negocio `getFile`, que dado un identificador de fichero, recuperara su contenido. Dicho método debería ser un método complejo con argumentos de salida.
- Se podría definir un método de negocio `putFile`, que permitiera la adición de un fichero, o la modificación del mismo, en la tabla correspondiente. Dicho método debería ser un método complejo con argumentos de entrada.

Estilo/Uso

En el mundo de los servicios SOAP, se ha producido una evolución en la manera en que los datos son incluidos dentro del cuerpo de los mensajes SOAP, así como descritos en el WSDL.

Los servicios SOAP de Meta4 pueden generarse indistintamente como `RPC/Encoded` (modo clásico) o `Document/Literal-Wrapped` (modo recomendado).

Meta4 hace posible que un servicio se despliegue en cualquiera de estos dos modos, conviviendo en una misma instalación los dos tipos de servicios.

1. Cuando un servicio se exporta desde la consulta, se crea automáticamente en el modo `Document/Literal-Wrapped`.

Compatibilidad

Los servicios SOAP generados por Meta4 son compatibles con los clientes SOAP que puedan consumir servicios generados por AXIS 1.x. Para mejorar la

compatibilidad de estos servicios, como Axis 1.x presenta una limitación relativa al despliegue de ciertos objetos para el consumo de plataformas Microsoft, Meta4 incluye dentro de uno de los filtros desplegados en sus aplicaciones web una herramienta llamada FixWSDL que modifica al vuelo el WSDL generado por Axis 1.x para mejorar su compatibilidad.

Véase más información en la sección “Configuración del filtro SOAP”.

Características de Meta4 como cliente SOAP

Meta4 como cliente SOAP es capaz de enviar mensajes de entrada a un servicio que interprete el protocolo SOAP de mensajería, y de recibir mensajes de salida en dicho protocolo, de una forma que no requiere el uso de programación específica para construir dichos mensajes.

En la actual versión de la tecnología de Meta4, la forma recomendada de invocar a los servicios SOAP externos es utilizando la tecnología de invocación de Java. Con esta tecnología, a través de la herramienta Axis WSDL2Java, se generan las clases cliente para poder invocar a un servicio. El desarrollador, una vez disponibles dichas clases generadas, las puede invocar mediante un API que le permite programar como si codificara en Java, pero en el lenguaje LN4.



En versiones anteriores, se utilizaba una arquitectura para conectarse con servicios SOAP basada en la tecnología de Meta4 de "conector genérico". Esta arquitectura imponía restricciones al formato de los datos, además de utilizar una componente de terceros para SOAP que no se mantenía al día de las especificaciones más recientes, por lo que no se recomienda su utilización a partir de la versión actual.

Servicios SOAP que pueden consumirse desde Meta4

En tiempo de ejecución las limitaciones de ejecución de los servicios serán las mismas que las de la conocida plataforma Axis 1.1.

Meta4 como servicio SOAP: aspectos prácticos

2

Acerca de este capítulo

En este capítulo se explica cómo generar y publicar un servicio SOAP de Meta4. El proceso de creación de un servicio SOAP desde la aplicación de Meta4 implica cuatro fases:

1. Instalar y configurar las componentes de la aplicación necesarias para la publicación, generación y ejecución de los servicios SOAP.
2. Crear los métodos de negocio del Meta4Object en cuestión y publicarlos, mediante una presentación disponible a tal efecto.
3. Proceder a la generación de código, compilación y activación del servicio, Esto se puede realizar de modo automático al acceder al servicio, ya que existe un filtro que detecta las peticiones de WSDL y ejecución y ordena la generación de servicios que desean usarse sin encontrarse en disco, o bien se puede realizar de forma manual, utilizando unas páginas JSP diseñadas específicamente para controlar los servicios de Meta4 exportados en un servidor web.
4. Llamar al servicio exportado.

Instalación de componentes necesarias

Para poder desarrollar nuevos servicios SOAP, debe instalar los siguientes elementos:

- **Cliente de desarrollo:** Se necesita instalar un cliente de desarrollo para publicar los servicios SOAP.
- **Cliente ligero:** Se necesita un contenedor Web, servidor Web y con aplicación Web desplegada y servidor de aplicaciones conectado al repositorio de Meta4, para generar y ejecutar los servicios SOAP. Consulte el capítulo Instalación de la aplicación Web en Windows o Instalación de la aplicación Web en UNIX del *Guía de Instalación*. En particular, debe verificar los siguientes tres puntos en su instalación:
 - El primero es tener tools.jar en el classpath, para que sea posible realizar la compilación de ficheros java. Para que en la máquina esté disponible el fichero tools.jar, es necesario que esté instalado el entorno de desarrollo de Java (JDK), no sólo el entorno de Runtime (JRE) de la versión certificada.

EJEMPLO

En JRun, verifique que el archivo jvm.config contiene una referencia a tools.jar, como sigue:

```
# java.class.path - use this for adding individual jars or
# directories. When directories are included they will be searched
# for jars and zips and they will be added to the classpath (in
# addition to the directory itself), the jar to be used in
# launching
# will be appended to this classpath
java.class.path={application.home}/servers/lib, C:\\Archivos de
Programa\\Meta4\\M4WS\\default\\WEB-INF\\classes, C:\\Archivos de
Programa\\Meta4\\M4WS\\default \\WEB-INF\\lib, {java.home}/lib/
tools.jar
```

- El segundo, si desea que el contenedor no se reinicie cuando se generen servicios nuevos, es deshabilitar todas aquellas opciones en el contenedor web que causen la recarga del contenedor. En una sección posterior se detallarán las consecuencias de esta medida.

EJEMPLO

En JRun, para evitar que se reinicie Jrun cada vez que se active un servicio (por ejemplo, en una máquina donde estén trabajando varios desarrolladores), edite jrun_root/servers/jrun_server/SERVER-INF/jrun.xml y desactive la opción HotDeploy del DeployerService como se indica en el siguiente fragmento de código. HotDeploy reinicia automáticamente un módulo al detectarse cambios

físicos en uno de los archivos del módulo o de los descriptores de despliegue en directorios expandidos. Así, por ejemplo, cuando se modifica el archivo `jrweb.xml` en un directorio expandido, se lleva a cabo el despliegue en caliente de un archivo WAR.

```
<service class="jrun.deployment.DeployerService"
name="DeployerService">
<attribute name="HotDeploy">false</attribute>
```

-
- Opcionalmente, si desea realizar la generación de servicios siguiendo el procedimiento manual, el tercero es disponer de un usuario con rol `SHCO_ADV_DEVELOPER`, para poder localizar a través de opciones de menú las páginas de administración de servicios SOAP.
 - Por último, si desea realizar la generación de servicios siguiendo el procedimiento automático, además de verificar que el filtro de autogeneración está presente (`M4SOAPAutoGenerationFilter` en `web.xml`), y activado (`enabled=true`), debe asegurarse de que el usuario `WEBADM` está correctamente configurado (por defecto sale con la contraseña `M4ADM`, pero se recomienda que realice un cambio).

El usuario `WEBADM` es un usuario de la aplicación cuya utilización se describe en el manual **Administración de las Aplicaciones Web**. Para comprobar rápidamente si está configurado, se puede utilizar la página `http://website:puerto/config/espanol/webappconfig.jsp`. Si `WEBADM` está configurado, se mostrará una pantalla de autenticación al acceder a la página. Para verificar la contraseña, nos autenticamos para entrar en la página.

Páginas de configuración web de Meta4	
Usuario de configuración	WEBADM
Contraseña de configuración	*****
Enviar Cerrar	

En caso de que no estuviera configurado, basta con entrar en la opción "securizar sitio" para crear el fichero, con el usuario y la contraseña deseada.

Parámetros de inicio	
Usuario de inicialización	WEBADM Securizar Sitio (Crear o actualizar fichero encriptado)
Fichero encriptado de inicialización	D:/0/m4tech-6.0/m4ws4j_tomcat/WEB-INF/classes/properties/webadmpass.bin

En caso de que estuviera configurado, la contraseña local debe coincidir con la que se encuentra almacenada en la base de datos.

Recomendaciones de seguridad para la instalación de las aplicaciones de Meta4

Usuarios de arranque

Existen usuarios de la base de datos con una contraseña predeterminada, para la realización de tareas administrativas. Estos usuarios vienen con una contraseña por defecto que, por seguridad, es conveniente modificar lo más pronto posible.

Un procedimiento para securizar las contraseñas de estos usuarios sería el siguiente:

1. Instale la base de datos.
2. Con un cliente de desarrollo, entre a cambiar la contraseña de los usuarios M4ADM y WEBADM.
3. Instale el servidor o los servidores de aplicaciones, con la contraseña cambiada de M4ADM.
4. Instale el servidor web con la aplicación web de Meta4. Si la contraseña está cambiada, no se harán las operaciones iniciales en la precarga del servlet. Entre en las páginas de configuración con la contraseña por defecto, y cámbiela a la nueva contraseña. Verifique que la precarga se hace en el inicio. Verifique también que la autogeneración de servicios SOAP funcione.

Creación y publicación de los métodos de negocio

En este apartado se describe cómo crear y publicar un método de negocio.

Creación del objeto de negocio con sus métodos

1. Cree el objeto de negocio que implemente la funcionalidad. Este será un Meta4Object que puede utilizar estructuras de nodo existentes o nuevas.

EJEMPLO

Si desea desarrollar un método que obtenga, para un país dado, un listado de sus provincias, ya existe un Meta4Object llamado SCO_GEO_STRUCTURE, que se puede utilizar como objeto de negocio.

Pero si no desea modificarlo, podría crear un Meta4Object copiado de éste, llamado SOAP_SCO_GEO_STRUCTURE_SOAP, con estructura análoga al anterior, en el que añadirá los métodos de negocio que necesite.

-
2. Identifique qué operaciones o métodos de negocio deben exportarse.

Cada método de negocio estará asociado a un método LN4. Si no existe ningún método LN4 apropiado para exportar la funcionalidad deseada, deberá crearlo.

EJEMPLO

Cree un método nuevo, GET_COUNTRY_INFO, con retorno de tipo Number, modo Ambas partes (sinc.), y un argumento AI_ID_COUNTRY de tipo STRING. Para que el método, dado un identificador de país, cargue las provincias, el código de la regla asociada al método debe ser:

```
sSQLString = "FROM &STD_COUNTRY A WHERE A.STD_ID_COUNTRY LIKE  
? (2,4,0) "
```

```
SYS_SENTENCE_A = sSQLString  
SYS_PARAM_A = ID_COUNTRY + "%"
```

```
DESTROYBLOCK()  
ROOTLOAD()  
return 1
```

Dar de alta los métodos de negocio

El siguiente paso es dar de alta los métodos de negocio que se acaban de implementar.

1. Abra la presentación desde la que se realiza esta operación, para ello, existen las opciones que se describen a continuación.
 - En el cliente de desarrollo, seleccione la opción de menú Herramientas de desarrollo | Diseñador de Meta4Object, y abra el Meta4Object que quiera exportar.
 - En el Diseñador de Meta4Objects, seleccione la opción de menú **Herramientas | Ir a Servicios SOAP**.

En esta presentación debe completar los datos del método del negocio, que contendrá la definición del elemento que quiera exportar, de acuerdo con las siguientes instrucciones:

Figura 3. Métodos de negocio del Meta4Object

ID Meta4Object	ID Estilo/Uso	ID Seguridad Web Services
SCO_GEO_STRUCTURE_SOAP	0	0

ID Método de negocio	ID Nodo	ID Elemento	Método de negocio	Descripción método de negocio
GET_COUNTRY_INFO	STD_COUNTRY	GET_COUNTRY	GET_COUNTRY_INFO (SOAP)	

ID Nodo argumento	ID Tipo de argumento
STD_COUNTRY	Salida

2. Por defecto, indique los valores de Estilo/Uso, Seguridad Web Services con valor a 0.
3. Seleccione en el campo ID Método de Negocio, un nombre para el método de negocio. Este será el nombre del método que invocará desde SOAP, no el nombre del ítem LN4. El nombre del método de negocio no puede ser una palabra reservada de Java.
4. Seleccione el nodo donde está el método LN4 que se exporta como método de negocio, así como el nodo, la estructura del nodo y el nombre de este método.

-
5. Añada en la pestaña inferior, tantos nodos de entrada y de salida como sean necesarios y grabe los cambios.

EJEMPLO

Siguiendo con el ejemplo de las provincias, en la presentación de alta de métodos de negocio, dé de alta un nuevo método de negocio con identificador `getCountryInfo`.

Indique que se corresponde con el método `GET_COUNTRY` del nodo `STD_MT_COUNTRY` del Meta4Object `SOAP_SCO_GEO_STRUCTURE`.

En la pestaña argumentos de tipo nodo, añádale un argumento complejo de salida (o nodo de salida), que es el nodo raíz `STD_MT_COUNTRY`.

Procedimiento de generación y activación de servicios

En este apartado se describe cómo conseguir que el servicio SOAP esté disponible en el servidor web de Meta4.

Como se mencionó anteriormente, existen dos procedimientos para generar y activar servicios SOAP de Meta4:

- **Procedimiento manual:** Se accede a una página de administración desde la que se puede consultar cuántos servicios están disponibles para ser generados, y desde la que se puede iniciar el proceso de generación, compilación y activación.
- **Procedimiento automático:** Al acceder al servicio (para solicitar su WSDL o su ejecución) se generará, compilará y desplegará automáticamente su código. Para optar por esta forma de generación, es necesario conocer el identificador del servicio.

Procedimiento manual desde las páginas de administración

En esta sección se describe cómo activar un método de negocio desde la página de administración de servicios SOAP. Inicialmente no habrá código generado por lo tanto la activación realizará primero la generación y compilación del código Java asociado al método de negocio. Si la activación se ha realizado con éxito el servicio SOAP estará disponible en el servidor Web. Después podrá invocarse al método de negocio desde cualquier cliente SOAP.

1. Inicie la aplicación web de Meta4 de cliente ligero y establezca una sesión en ella. Consulte el manual de administración de servicios web si necesita ayuda en esta tarea.
2. Diríjase a la página de administración de servicios SOAP tras obtener una sesión web válida, de una de las dos siguientes maneras:
 - Acceda la página Diseñador de servicios SOAP desde la opción de menú **Herramientas Avanzadas | Diseñador de servicios SOAP** que puede encontrar en la página de portal (como se indicó anteriormente, sólo disponible si tiene el rol SHCO_ADV_DEVELOPER).
 - Teclee directamente la dirección de la página, por ejemplo:

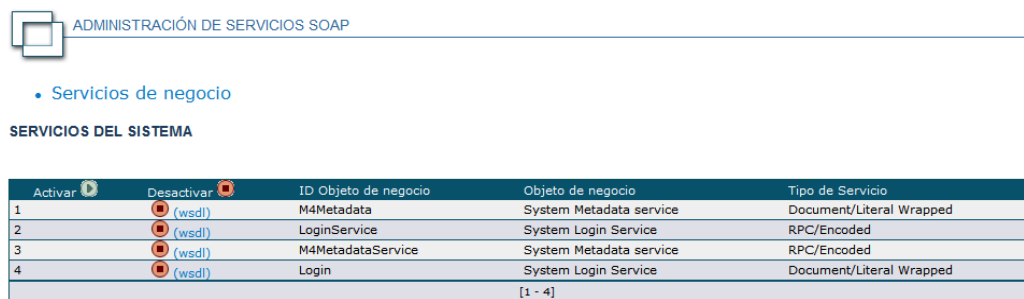

```
http://servidor:puerto/servlet/CheckSecurity/JSP/tcadmin/
soapservices/sch_bo_method.jsp
```

3. Verifique si tiene el servicio de autenticación (login) activado desde la página de Administración de servicios SOAP, especialmente si es la primera vez que intenta exportar servicios SOAP de Meta4 en el entorno Web. Para ello, utilice una de las dos maneras que se indican a continuación:
 - Consultando el listado de servicios que aparece si se introduce la URL `http://servidor:puerto/services`.
 - Accediendo a la opción **Servicios del Sistema** disponible en la página Administración de Servicios SOAP.
 - Editando el archivo `server-config.wsdd` como se describe en la sección “Configuración de `server-config.wsdd`”.

Igual que los servicios SOAP de Meta4 pueden generarse indistintamente como Document/Literal-Wrapped (modo recomendado) o RPC/Encoded (modo clásico), el servicio de autenticación se proporciona en dos variantes, cuyo identificador de servicio es Login (Document/Literal-Wrapped) o LoginService (modo clásico). Se recomienda la utilización del servicio Login puesto que presenta la máxima compatibilidad con clientes SOAP de distintos proveedores de software.

Si el servicio de autenticación no apareciera activado, es necesario activarlo haciendo clic en el botón **Activar**. Se sugiere comprobar a continuación en `http://servidor:puerto/services` que este el servicio de autenticación (login).

Figura 4. Página de administración de servicios SOAP



Activar	Desactivar	ID Objeto de negocio	Objeto de negocio	Tipo de Servicio
1	(wsdl)	M4Metadata	System Metadata service	Document/Literal Wrapped
2	(wsdl)	LoginService	System Login Service	RPC/Encoded
3	(wsdl)	M4MetadataService	System Metadata service	RPC/Encoded
4	(wsdl)	Login	System Login Service	Document/Literal Wrapped

[1 - 4]

4. Proceda al paso crucial que es la generación de código e instalación del servicio.

Para ello, en la página Servicios de Negocio busque o indique el nombre del objeto de negocio (el Meta4Object que exporta sus métodos por SOAP) y haga clic en el botón **Activar**.

En esta página sólo podrá activar los servicios de negocio que previamente haya publicado en la presentación de alta de métodos de negocio.

EJEMPLO PROVINCIAS

En el ejemplo de las provincias, filtre por los servicios de negocio que comienzan por SOAP y obtendrá SOAP_SCO_GEO_STRUCTURE. Haga clic en el botón **Activar** hasta que desaparezca el botón **Activar** y ya sólo se pueda **Desactivar**.

Figura 5. Página de los servicios de negocio



ADMINISTRACIÓN DE SERVICIOS SOAP

• Servicios del sistema

SERVICIOS DE NEGOCIO

Filtre servicio de negocio según estado...
 Todos ☒ Activos ☐ Inactivos ☐
 Seleccione el servicio de negocio
 ID Servicio de negocio

	Regenerar código	Activar	Desactivar	ID Objeto de negocio	Objeto de negocio	Tipo de Servicio
1				SNCO_AD_IC_KNWLD_LV_TRANSLATOR	Knowledge Level Transalator	RPC/Encoded
2				SNTC_ADB_VIEW_ALERT	Visualizar alertas	RPC/Encoded
3				SNTC_AD_MANAGERS	Poblacion de managers	RPC/Encoded
4				SNTC_AD_POPULATION	Poblaciones	RPC/Encoded
5				SNTC_GRAPH_BASE_TEMPLATE	Plantilla base para gráficos	RPC/Encoded
6				SNTC_SEC_OPT	SNTC_SEC_OPT	RPC/Encoded
7				SNTC_TIMEOUT	Gestión de timeout en .NET	Wrapped Document/Literal
8				ZZ_JESUS_NET	Canal de personas	RPC/Encoded

[1 - 8]

- Verifique, al finalizar, que el método ha sido activado en `http://servidor:puerto/services`. También puede verificar que el código ha sido generado buscando los ficheros `*.java` y `*.class` dentro de la aplicación web.

EJEMPLO

En el ejemplo de las provincias, si su aplicación Web ha sido instalada en `C:\Archivos de Programa\Meta4\M4WS\default\`, los ficheros `*.java` y `*.class` se encontrarán en `C:\Archivos de Programa\Meta4\M4WS\default\WEB-INF\classes\com\meta4\soapservices\services\rpc\soap_sco_geo_structure`.

Terminado el ejemplo con éxito, el detalle de los métodos de negocio exportados sería el siguiente:

- AdminService ([wsdl](#))
 - AdminService
- Version ([wsdl](#))
 - getVersion
- Login ([wsdl](#))
 - login
 - logout
 - loginWithRole
 - loginWithCertificate
 - retrieveM4Session
- SOAP_SCO_GEO_STRUCTURE ([wsdl](#))
 - GETCOUNTRYINFO
 - retrieveM4Session



Si existiera algún error en la activación o desactivación, el icono asociado parpadeará y aparecerá una ventana de error indicando qué objetos de negocio han fallado. Si la activación falla, puede intentar la regeneración de código. Si en la regeneración ocurriera un error, se creará un directorio con el código generado. El error más frecuente es que no se puede compilar el código generado porque falta el tools.jar en el CLASSPATH.

EJEMPLO

Según las rutas indicadas anteriormente, el directorio con el código correspondiente a la situación de error se encontraría en

```
C:\Archivos de Programa\Meta4\M4WS\default\WEB-INF\classes\com\meta4\soapservices\services\rpc\soap_sco_geo_structure.err
```



Al activar y/o desactivar un servicio SOAP desde la pantalla de administración de servicios SOAP, se resetea el contenedor Web y, por tanto, la página y el resto de servicios quedan inutilizables durante el tiempo en que se levanta de nuevo el contenedor Web.

Configuración del despliegue de los servicios

Configuración de server-config.wsdd

El administrador del sistema puede intervenir manualmente en el despliegue de los servicios, mediante la edición directa del archivo WEB-INF/server-config.wsdd. Siempre que se modifique este archivo, será preciso reiniciar el contenedor web.

El archivo contiene una configuración global que puede ser preciso revisar, como por ejemplo, el directorio temporal de adjuntos (attachments.Directory), adaptándolo a la plataforma correcta:

■ En Windows

```
<globalConfiguration>
<parameter name="attachments.Directory" value="c:\m4ws4j\WEB-
INF\attachments"/>
```

■ En UNIX

```
<globalConfiguration>
<parameter name="attachments.Directory" value="/opt/m4ws4j/WEB-INF/
attachments"/>
```

Cada servicio tiene una configuración individual que también puede ser necesario comprobar. Por ejemplo, el servicio de autenticación Login debe contener un código XML similar al siguiente:

EJEMPLO

```
<service name="Login" provider="java:RPC" style="wrapped"
use="literal">
  <parameter name="allowedMethods" value="*"/>
  <parameter name="scope" value="session"/>
  <parameter name="className"
value="com.meta4.soapservices.services.loginservice.LoginService"/>
  <typeMapping
deserializer="org.apache.axis.encoding.ser.BeanDeserializerFactory"
encodingStyle="" qname="ns1:M4loginOutput"
serializer="org.apache.axis.encoding.ser.BeanSerializerFactory"
type="java.com.meta4.soapservices.services.loginservice.M4loginOutput"
xmlns:ns1="http://schemas.meta4.com"/>
  <namespace>http://schemas.meta4.com/</namespace>
</service>
```



Es importante prestar atención al namespace de este servicio: para maximizar la compatibilidad con clientes SOAP (por ejemplo de Microsoft), el namespace debe incluir la barra final (<http://schemas.meta4.com/>).

Configuración del filtro SOAP

Como se indicaba en secciones anteriores, en web.xml se configura el filtro SOAP. Este filtro (M4SOAPAutoGenerationFilter) sirve para realizar la generación de servicios siguiendo el procedimiento automático, si está activado (enabled=true). Recuerde que para realizar la generación automática, el usuario WEBADM debe estar configurado para apuntar a un archivo encriptado de contraseña válido.

El filtro tiene una funcionalidad adicional, que no se puede desactivar, que es la de realizar la operación FixWSDL para proporcionar compatibilidad con clientes SOAP.

Tenga en cuenta que si elimina completamente el filtro no se ejecutará la funcionalidad FixWSDL necesaria para generar WSDL comprensibles por algunos clientes, como por ejemplo Microsoft InfoPath. Por lo tanto, al borrar de web.xml el filtro se generan WSDL diferentes.

Para poder aplicar la funcionalidad FixWSDL en Tomcat y evitar que ocurra un conflicto entre la librería JAR interna que posee la instalación de Tomcat y la que incorporan los servicios Web de Meta4 (archivo 3rdparty-all.jar), provocado por un error de la componente XALAN que se utiliza para convertir un WSDL en otro, copie el archivo "xalan-2.3.1.jar" dentro de la carpeta "endorsed", regenerando el WSDL.

Procedimiento automático con identificador de servicio

Para que pueda realizarse la generación automática de un servicio, debe conocer detalles sobre él, como su identificador. En ese caso, basta con que introduzca en el navegador web una URL de solicitud de WSDL o emitir una petición de ejecución para que el sistema comience a determinar si debe generar un nuevo servicio.

EJEMPLO

http://servidor:puerto/services/SOAP_SCO_GEO_STRUCTURE?wsdl

Si introduce una URL de petición de WSDL o de ejecución, el sistema comprueba si el servicio en cuestión existe o no. Si es la primera vez que accede al sistema de autogeneración, el filtro se autentica como WEBADM para comprobar la versión del servicio en la base de datos, guardando la sesión para futuras operaciones.

Si lo que se ha solicitado es el WSDL o la ejecución de un servicio que ya se ha generado en el disco, se ofrece el WSDL en caso de que la versión no haya cambiado. Si pedimos un servicio que no existe en el repositorio, el filtro ve que no existe, nos da la pantalla de "no hay WSDL". Por último, si la versión del servicio que tenemos en disco ha cambiado, o bien no lo tenemos en el disco pero sí en la base de datos, el sistema genera automáticamente el código Java correspondiente.

En el caso de que se desee eliminar un servicio SOAP del servidor web de Meta4, se recomienda borrar completamente el directorio con el código y descriptores de despliegue que se genera en el disco.

EJEMPLO

Según las rutas indicadas anteriormente, el directorio a borrar en caso de desear eliminar un servicio sería

```
C:\Archivos de Programa\Meta4\M4WS\default\WEB-INF\classes\com\meta4\soapservices\services\rpc\soap_sco_geo_structure
```

Llamar al servicio exportado

Una vez creado del objeto de negocio, dado de alta, generado y compilado el código, el servidor Web se comportará como servidor SOAP. En este apartado se describen aquellas características de los servicios SOAP de Meta4 que es necesario conocer para poder llamarlos.

Es imprescindible saber que para llamar a un servicio SOAP de Meta4 es necesario llamar previamente al método **login** del servicio de autenticación, y al finalizar la llamada, es recomendable llamar al método **logout** del mismo.

Como en general, para llamar cualquier servicio SOAP, las aplicaciones cliente SOAP suelen aceptar un archivo WSDL del que deducen cuáles son los métodos exportados, es conveniente saber que para obtener el fichero WSDL asociado a un servicio SOAP de Meta4, puede procederse de dos maneras:

- En la página Servicios de Negocio haga clic en el botón **Detalle de métodos de negocio**.
- Directamente, indicando el nombre del servicio con el parámetro WSDL.

EJEMPLO

`http://servidor:puerto/services/SOAP_SCO_GEO_STRUCTURE?wsdl`

Normalmente, y como ya se ha indicado, para crear un cliente SOAP se utiliza un toolkit que a partir de WSDL genere el tipo de cliente más adecuado para la plataforma en cuestión. Pero si lo único que desea es verificar que un servicio SOAP funciona y no dispone de un toolkit apropiado, lo más sencillo es disponer de un mensaje SOAP de ejemplo y enviarlo con uno de esos clientes que aceptan un SOAP como parámetro de entrada, como AxisSoapClient.

EJEMPLO

Si desea solicitar al servicio de negocio SCO_SOAP_GEO_STRUCTURE las provincias de España (con ID 034), utilizando el cliente AxisSoapClient, en la plataforma Windows, en la que se distribuyen unos ficheros de comandos idóneos para pequeñas pruebas de los servicios web exportados por Meta4.

-
1. Modifique (raiz)\WEB-INF\usertools\admin\environment.bat para que apunte a su servidor web (host y puerto), así como al ejecutable Java.
 2. Modifique (raiz)\WEB-INF\usertools\client\samples\login\login.xml para que contenga usuario y contraseña
 3. Cree un nuevo archivo llamado (raiz)\WEB-INF\usertools\client\samples\sco_geo_structure\getcountryinfo.xml como en el siguiente ejemplo:

EJEMPLO

```
<?xml version='1.0' encoding='ISO-8859-1'?>
<SOAP-ENV:Envelope
  xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema">
  <SOAP-ENV:Body>
    <ns1:GETCOUNTRYINFO
      SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/"
      xmlns:ns1="SOAP_SCO_GEO_STRUCTURE">
      <ai_id_country xsi:type="xsd:string">034</ai_id_country></
      ns1:GETCOUNTRYINFO>
    </SOAP-ENV:Body>
  </SOAP-ENV:Envelope>
```

4. Ejecute el archivo de comandos (raiz)\WEB-INF\usertools\client\axissoapclient.bat con el fichero de login y de getcountryinfo como parámetros. Se recomienda redirigir la salida para ver el mensaje SOAP de vuelta.

EJEMPLO

```
(raiz)\WEB-INF\usertools\client>axissoapclient  
samples\login\login.xml  
samples\soap_sco_geo_structure\input_getcountryinfo.xml >  
samples\soap_sco_geo_structure\output_getcountryinfo.xml
```

Si ve correctamente las provincias de España, la ejecución ocurrió con éxito.

Meta4 como cliente SOAP: aspectos prácticos

3

Acerca de este capítulo

En este capítulo, se explican las formas de invocar un servicio SOAP desde una aplicación de Meta4.

Invocar servicios SOAP a través de Java

Para poder invocar un servicio SOAP desde Java, debe seguir estos pasos en el orden indicado:

1. Configurar la máquina virtual de Java.
2. Crear un cliente SOAP.
3. Crear un Meta4Object para llamar a las clases correspondientes para invocar el servicio.

Configuración de la máquina virtual Java

Para la generación de clientes SOAP e invocación de servicios SOAP mediante Java es necesario configurar las aplicaciones de Meta4 para ubicar la máquina virtual de Java.

1. Cliente de desarrollo: es necesario un JDK si se van a crear clases cliente. Si el JDK está instalado, la configuración se realiza de forma automática en la herramienta de configuración del cliente, que determina la ruta al JRE de dicho JDK, y la ruta al fichero tools.jar, que se incluye en el fichero XML de configuración del registro. Por ejemplo: C:\Program Files\Java\jre1.6.0_07\lib\tools.jar.
2. Servidor de aplicaciones: esta configuración cubre los casos de ejecución de servicios SOAP desde clientes Rich Web y desde aplicaciones Web para clientes HTML. Simplemente se necesita un JRE para ejecutar las clases Java. Recuerde que dispone de una versión del JRE en el CD de producto.

La configuración se debe realizar desde el editor de configuraciones, pestaña "Third Party", y los valores a configurar son:

M4JAVAHOME: ruta donde se ubica la instalación de JRE

- Ejemplo en servidor Windows: C:\Program Files\Java\jre1.6.0_07
- Ejemplo en servidor Unix: /usr/java/jre ó /usr/java/jre16

M4JAVAARGS: argumentos opcionales que pasar a la máquina virtual Java, por ejemplo, memoria máxima, etc.

Creación de un cliente SOAP

A continuación es necesario definir un cliente SOAP. Para ello, vaya a la opción de menú Herramientas de desarrollo | Meta4Objects | Definición clientes SOAP, que permite dar de alta y/o consultar los clientes SOAP que posteriormente queremos invocar desde un M4Object. Desde esta opción de menú, se puede acceder al mantenimiento de los clientes SOAP definidos y sus versiones registradas en el sistema.

Cada uno de los clientes SOAP definidos se identifican a través del identificador y el nombre del cliente SOAP. Una vez dentro de la presentación de Mantenimiento de clientes SOAP, en la pestaña **Clientes SOAP**, utilice los campos **ID Cliente SOAP**, **Nombre** y **Descripción** para dar de alta un cliente SOAP. En el árbol de contenidos de la presentación, también se pueden ver las versiones definidas para cada cliente SOAP. Para ello, haga clic en la pestaña **Versiones**, que contiene los siguientes campos:

- **ID Versión:** código de identificación autonumérico asignado por el sistema a cada versión de un cliente SOAP.
- **Descripción del servicio:** campo opcional para introducir la ruta de acceso al fichero wsdl que describe el servicio SOAP. A la derecha del campo de descripción del servicio dispone del botón **Seleccionar el fichero WSDL**, que le permitirá seleccionar dicha ruta.
- **Versión Java:** versión de la máquina virtual Java con la que se va a generar el cliente del servicio SOAP. Este campo será rellenado por el sistema cuando se genere el cliente (cuando se salve la presentación).
- **Versión Axis:** versión de Axis (Apache) con la que se va a generar el cliente del servicio SOAP. Este campo será rellenado por el sistema cuando se genere el cliente (cuando se salve la presentación).

Utilice los campos de esta pestaña para dar de alta una versión para el cliente que ha creado, para lo que deberá especificar un fichero WSDL y guardar la presentación. Cuando guarde la presentación, se generará un fichero empaquetado de clases Java (jar) que contendrá las clases cliente para poder invocar al servicio definido por el fichero WSDL indicado. Tanto el fichero WSDL como el archivo jar se guardan en el repositorio.

En este proceso también se guarda la versión de máquina virtual de Java y la versión de Axis utilizada para generar el archivo jar.

En la parte inferior de la pestaña de datos de las versiones hay un cuadro resumido de las mismas, donde se indica el identificador de cada una, la descripción del servicio y las versiones Java y Axis correspondientes. Puede utilizar este cuadro para desplazarse por las diferentes versiones, al igual que los botones de navegación superiores.

Por último, debajo del árbol de contenidos, encontramos dos acciones asociadas a las versiones:

- El botón **Descargar jar** nos permitirá descargar a un directorio local el fichero jar que contiene el cliente generado para el servicio SOAP.
- El botón **Ver fichero WSDL** nos permitirá visualizar el fichero de definición del cliente SOAP (fichero WSDL), si se ha especificado alguno.

Creación de un Meta4Object para llamar a las clases correspondientes

Una vez definido un cliente SOAP, se pueden llamar las clases correspondientes para invocarlo, para lo que es necesario tener configurada la máquina virtual de Java según los pasos especificados en la sección Configuración de la máquina virtual de Java.

Para poder crear/destruir objetos Java e invocar métodos de objetos previamente creados, se ha establecido la estructura de nodo tecnológica SCH_JAVA_API, que contiene toda la funcionalidad necesaria para ello.

En esta estructura de nodo, se han definido los siguientes métodos:

- **LOAD_CONTEXT**: este método permite cargar el contexto necesario para poder tener disponibles las clases que se van a invocar. El método tiene los siguientes argumentos:
 - **AI_ID_JAR**: identificador del archivo jar/cliente SOAP.Como segundo argumento podemos especificar una versión para un cliente SOAP definido o fichero jar externo. En este caso, en vez de cargar la última versión, se carga la versión especificada.
- **CREATE_OBJECT**: este método permite crear un objeto Java. Para ello se deben especificar los siguientes argumentos:
 - **AI_SCLASS_NAME**: nombre de la clase Java de la que se quiere crear un objeto.
 - **AO_IRETURN**: indica el retorno del proceso de creación del objeto Java.

En este método también es posible especificar como argumento de entrada los parámetros del constructor de la clase, si los tuviere. Como resultado de la ejecución del método se obtiene un número que representa un handle al objeto Java creado.

- **INVOKE_METHOD**: este método permite invocar a un método de un objeto Java previamente creado. Para ello este método tiene los siguientes argumentos:
 - **AI_OBJ_HANDLE**: handle al objeto Java previamente creado, sobre el que se desea ejecutar el método.
 - **AI_SMETHOD_NAME**: nombre del método Java.

-
- **AO_IRETURN:** indica el retorno del proceso de invocación del método Java.

Si el método Java tiene argumentos, se los tendremos que pasar como sucesivos argumentos al método **INVOKE_METHOD**. Como resultado de la ejecución del método **INVOKE_METHOD** se obtiene el resultado de la ejecución del método Java (retorno del método Java).

- **DESTROY_OBJECT:** este método permite destruir un objeto Java y liberar los recursos asociados a su creación. Para ello se debe especificar como argumentos:
 - **AI_OBJ_HANDLE:** handle al objeto Java previamente creado.

Los servicios SOAP de Meta4 en detalle

4

Acerca de este capítulo

En este capítulo se profundiza, mediante ejemplos, en el desarrollo, seguridad y ciclo de vida de los servicios SOAP de Meta4.

Ciclo completo de desarrollo y ejecución de un servicio SOAP con operaciones de consulta y persistencia

En esta sección se describe cómo crear un servicio SOAP que haga posible realizar operaciones de consulta y modificación de datos.

En general, para crear un servicio SOAP de Meta4 deben seguirse estos pasos:

1. Planificar las funcionalidades deseadas
2. Crear el objeto de negocio, así como los métodos LN4 necesarios para implementarla.
3. Publicar los métodos de negocio
4. Obtener descripción del servicio
5. Crear las clases proxy
6. Elaborar el código cliente
7. Realizar verificación del servicio
8. Planificar las funcionalidades

Se desea consultar y modificar la información de las personas en la aplicación de Meta4. Lo primero será estudiar la interfaz a publicar.

Un método de **consulta de personas** puede tener las siguientes características:

- **Entrada:** Identificador de empleado (e.g. 10028) o parte del mismo (e.g. 100)
- **Salida:** Lista de personas cuyo identificador contiene al menos parte del recibido, posiblemente vacía. Para cada persona se informará de su nombre, fecha de nacimiento, etc. así como su lista de correos electrónicos (casa, trabajo, etc.). También será deseable un retorno con éxito o fracaso, y si es necesario, una lista de errores asociados.

Un método de negocio para la **modificación de personas** puede tener las siguientes características:

- **Entrada:** Se introducirán datos que identifican a una persona. En el caso de que la persona exista, se desea que dichos datos se modifiquen. En el caso de que la persona no exista, se desea que se cree dicha persona en la base de datos con todos los datos indicados.
- **Salida:** Un retorno que indique éxito o fracaso, y si es necesario, una lista de errores asociados.

Crear objeto y métodos de negocio

Decidida la interfaz del servicio, se investigará la manera más eficaz de crear los métodos de negocio. En el caso de que solamente se deseara realizar una operación de lectura de datos, dentro de ciertas limitaciones, sería posible publicar rápidamente desde la herramienta de la Consulta. Para ello, se crearía una consulta sobre las tablas que se desean consultar y se indicaría que se exportará como servicio SOAP.

En caso de tratarse de operaciones de persistencia de datos o de lectura de datos que no cumplan los requisitos para ser exportadas por la consulta (la consulta produce Meta4Objects con nodos raíz sobre los que no se puede efectuar operaciones de escritura), se crearán a mano los métodos a exportar. Habrá que verificar si ya existe un Meta4Object que tenga los métodos que se desea exportar, o de lo contrario, deben crearse métodos nuevos.

En el ejemplo de las personas, se creará un nuevo Meta4Object (ZZ_TK_PERSON_SOAP), asociando su nodo raíz (ZZ_TK_PERSON_SOAP_ROOT) a la tabla de personas (STD_PERSON), definiendo un nodo hijo (ZZ_TK_PERSON_EMAIL) asociado a la tabla de correos electrónicos (STD_EMAIL). Se puede crear dicho Meta4Object con el asistente del Development Framework.

En el nodo raíz se añadirán dos métodos nuevos llamados:

-
- **SOAP_PUT_PERSON:** Método cuyo código simplemente efectúa la persistencia del nodo raíz. El comportamiento del método de negocio asociado no se basará en el código LN4 ejecutado sino en las características del argumento introducido, como se detallará a continuación.

•

```
Persist_Tree()  
return 1
```

- **SOAP_GET_PERSON:** Método cuyo código carga los registros necesarios en el nodo de salida. Después de crear el método, añada un argumento de entrada de tipo cadena variable para introducir el identificador de la persona (AI_ID_PERSON). En la regla asociada, cargue el nodo (LOAD_PRG). Para que solamente se carguen los registros filtrados por el identificador especificado por el identificador introducido como argumento, establezca una sentencia apropiada (SYS_SENTENCE).

•

```
sSQLString = "FROM &STD_PERSON A WHERE A.STD_ID_PERSON LIKE ?(2,4,0)  
  
SYS_SENTENCE_A = sSQLString  
SYS_PARAM_A = AI_ID_PERSON + "%"  
  
DestroyBlock()  
Rootload()  
return 1
```

Publicar los métodos de negocio

Una vez creados los métodos de negocio, en el menú del Diseñador de Meta4Objects, acceder a "Ir a Servicios SOAP" para realizar la publicación.

En primer lugar, se debe crear un nuevo registro para el Meta4Object ZZ_TK_PERSON_SOAP en la tabla de servicios de negocio. Esto indica que el Meta4Object tendrá métodos publicados. Además permite especificar sus

características. En ID Estilo/Usa, elija 0 si desea utilizar el modo clásico de Meta4 (RPC/Encoded), o 1 si desea utilizar el modo recomendado para interoperabilidad (Document/Literal-Wrapped). En ID Seguridad Web Services, elija 0 si desea utilizar la autenticación clásica, o en general si no desea implementar seguridad Username Token Profile, como se indicará en una sección posterior.

En segundo lugar, debe crear dos registros nuevos en la tabla de métodos de negocio. Estos dos nuevos registros se corresponderán con:

"putPerson: Añadir un método de negocio asociado al método LN4 SOAP_PUT_PERSON, con un argumento de tipo nodo de entrada (ZZ_TK_PERSON_SOAP_ROOT).

Servicios de negocio

Servicios de negocio

ID Meta4Object: ZZ_TK_PERSON_SOAP Person SOAP

ID Estilo/Usa: 1 ID Seguridad Web Services: 0

Métodos de negocio

ID Método de negocio: PUTPERSON

ID Nodo: ZZ_TK_PERSON_SOAP_ROOT

ID Elemento: SOAP_PUT_PERSON

Método de negocio: putPerson

Descripción método de negocio: Permitir que se añada una persona completa a la tabla de personas, se modifiquen sus : datos personales o se den de baja.

Argumentos de tipo nodo

ID Nodo argumento	ID Tipo de argumento	
ZZ_TK_PERSON_SOAP_ROOT	1	Entrada

- **getPerson:** Añadir un método de negocio asociado al método LN4 SOAP_GET_PERSON, con un argumento de tipo nodo de salida (ZZ_TK_PERSON_SOAP_ROOT)

Servicios de negocio

Servicios de negocio

ID Meta4Object: ZZ_TK_PERSON_SOAP Person SOAP

ID Estilo/Uso: 1 ID Seguridad Web Services: 0

Métodos de negocio

ID Método de negocio: SOAP_GET_PERSON

ID Nodo: ZZ_TK_PERSON_SOAP_ROOT

ID Elemento: SOAP_GET_PERSON

Método de negocio: getPerson

Descripción método de negocio: Dado un identificador de persona, retornar toda la información necesaria sobre la misma.

Argumentos de tipo nodo

ID Nodo argumento	ID Tipo de argumento	
ZZ_TK_PERSON_SOAP_ROOT	0	Salida

Obtener descripción del servicio

Una vez guardados los datos de los métodos de negocio, es necesario generar en el contenedor web el código Java necesario, compilarlo, desplegarlo en Axis, y obtener su descripción (WSDL). Esto es tan sencillo como iniciar el contenedor web y solicitar el WSDL.

http://tpbasexp.meta4.com:8100/services/ZZ_TK_PERSON_SOAP?wsdl

La petición de WSDL desencadena una petición de autogeneración, autocompilación y autodespliegue del código, que nos permite obtener instantáneamente su WSDL sin necesidad de codificar en Java ni de acceder a páginas especiales para desplegar el servicio.

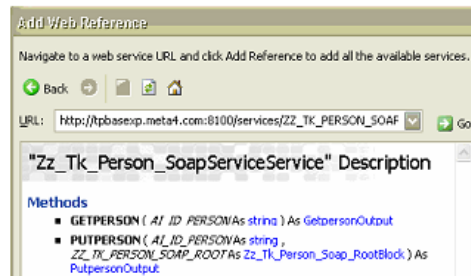
Crear las clases proxy

Para utilizar el servicio desde un cliente cualquiera, se necesita un desarrollador que a partir del WSDL, codifique las llamadas al servicio que estime convenientes. En primer lugar, el desarrollador debe crear a partir del WSDL unas clases cliente del servicio SOAP. Esto suele hacerse desde

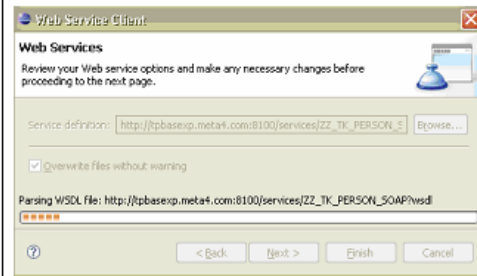
entornos de desarrollo preparados para crear automáticamente unas clases cliente, a las que se suele denominar clases Proxy del servicio. La forma de crear las clases Proxy depende del tipo de entorno de desarrollo.

En Visual Studio (.net) se debe crear una "Referencia Web" (botón derecho en la solución y "Add Web Reference"), mientras que en Eclipse (Java) se crea un "Web Service Client" (Menú New > "Other" > "Web Service Client").

C# (Microsoft VS .Net) > Add Web Reference



Java (Eclipse/Axis) > New > Web Service Client



Elaborar el código cliente

En un cliente SOAP, las llamadas a los métodos se realizan mediante objetos locales que representan el método remoto en las clases Proxy. A dichos objetos locales se les suele denominar "stubs".

Para obtener el "stub" de un servicio a partir de un código generado, se debe empezar por un código que depende del entorno de desarrollo:

"

;

- **C# (Microsoft VS .Net) > Obtener "stub" > A través de un objeto (*Service)**

```
Zz_Tk_Person_SoapServiceService oService = new Zz_Tk_Person_SoapServiceService();
```

Java (Eclipse/Axis) > Obtener "stub": A través de una factoría, se obtiene un localizador del servicio (*ServiceLocator) para construir una implementación del servicio, que puede ser necesario configurar.

```

static Zz_Tk_Person_SoapServiceServiceLocator oFactory = null;
static Zz_Tk_Person_SoapService oService = null;
static javax.xml.rpc.Stub oStub = null;

// 1. Instantiate a proxy factory
oFactory = new Zz_Tk_Person_SoapServiceServiceLocator();

// 2. Instantiate a proxy
oService = oFactory.getZZ_TK_PERSON_SOAP();

// 3. Cast to the interface (optional)
oStub = (javax.xml.rpc.Stub) oService;

```

Una vez que se dispone del objeto local para acceder el servicio, es necesario realizar la llamada al método, según el procedimiento que se resumen a continuación.

Código de cliente de métodos de negocio de servicios SOAP de Meta4 según el tipo de operación	
Leer datos (consulta)	Ejecutar el método y recoger el argumento nodo de salida, buscando en los resultados deseados.
Añadir registros	Construir un argumento nodo de entrada con la información necesaria y ejecutar el método de negocio.
Modificar registros	Realizar una operación de consulta sobre el nodo que contiene el registro en el que se realizará la modificación, obteniendo el identificador de registro (RID). Una vez obtenido el RID, construir un argumento nodo de entrada que contenga la información a modificar, así como dicho RID, e incluirlos en la llamada al método de negocio.
Borrar registros	Igual que en la modificación de registros, se debe realizar una operación de consulta sobre el nodo que contiene el registro en el que se realizará la modificación, obteniendo su RID. A continuación, construir un argumento nodo de entrada que contenga el RID, y la marca para borrado del mismo, e incluirlos en la llamada al método de negocio.

Ejemplo 1 (Leer Datos): En este ejemplo, escrito en C#, se listaría el identificador y el nombre de todas las personas cuyo identificador sea o empiece por un identificador dado. Por claridad, no se ha incluido el código de autenticación, pero este es imprescindible para el funcionamiento del servicio, como se indicará en una sección posterior.

"

```

using System;
using System.Collections.Generic;
using System.Text;
using zz_tk_person_soap_wrapped.com.meta4.tpbasexp;

namespace zz_tk_person_soap_wrapped
{
    class Zz_Tk_Person_Soap_Wrapped_main
    {
        static void Main(string[] args)
        {
            try
            {
                Zz_Tk_Person_SoapServiceService oService = new Zz_Tk_Person_SoapServiceService();

                /* Authentication code to be inserted here */

                Soap_Get_PersonOutput oOutputNode = oService.GETPERSON("M10128");

                Zz_Tk_Person_Soap_RootBlock oParentBlock = oOutputNode.zz_Tk_Person_Soap_Root;
                Zz_Tk_Person_Soap_RootRecord[] oParentRecordSet =
                oParentBlock.zz_Tk_Person_Soap_RootRecordSet;

                int iParentSize = oParentRecordSet.Length;

                for (int i = 0; i < iParentSize; i++)
                {
                    Console.WriteLine("Person ID " + i + ": " + oParentRecordSet[i].std_Id_Person);
                    Console.WriteLine("Person Name " + i + ": " + oParentRecordSet[i].
                    std_N_Family_Name_1);
                }

            }
            catch (Exception e)
            {
                Console.WriteLine("Error while running the service: ", e);
            }
        }
    }
}

```

Ejemplo 2 (Añadir registro): En este ejemplo, se toma un nuevo identificador de persona (STD_ID_PERSON, sería necesario que no estuviera en uso por una persona ya dada de alta en la base de datos), y se invoca el método para añadir nueva persona, incluyendo en el argumento de entrada sus datos y los datos de sus correos electrónicos.

"

```
/* Authentication code to be inserted before */

// Build input block/recordset.
Zz_Tk_Person_EmailBlock oNodeInputChild = new Zz_Tk_Person_EmailBlock();
Zz_Tk_Person_EmailRecord oRecordInputChild = new Zz_Tk_Person_EmailRecord();
oRecordInputChild.setStd_Email("alejandrog@meta4.com");
oRecordInputChild.setStd_Or_Mail(new Double(1));

GregorianCalendar std_Dt_Start = new GregorianCalendar(TimeZone.getTimeZone("GMT"));
GregorianCalendar std_Dt_End = new GregorianCalendar(TimeZone.getTimeZone("GMT"));

SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd HH:mm:ss");
sdf.setTimeZone(TimeZone.getTimeZone("GMT"));

std_Dt_Start.setTime(sdf.parse("2006-03-30 01:22:33"));
std_Dt_End.setTime(sdf.parse("2078-06-18 02:33:44"));

oRecordInputChild.setStd_Dt_Start(std_Dt_Start);
oRecordInputChild.setStd_Dt_End(std_Dt_End);

Zz_Tk_Person_EmailRecord[] oRecordsetInputChild = { oRecordInputChild };
oNodeInputChild.setZz_Tk_Person_EmailRecordSet(oRecordsetInputChild);

Zz_Tk_Person_Soap_RootBlock oNodeInputParent = new Zz_Tk_Person_Soap_RootBlock();
Zz_Tk_Person_Soap_RootRecord oRecordInputParent = new Zz_Tk_Person_Soap_RootRecord();

oRecordInputParent.setStd_Id_Person(sNew_Std_Id_Person);
oRecordInputParent.setStd_N_Family_Name_1("García");
oRecordInputParent.setStd_N_First_Name("Alejandro");
oRecordInputParent.setSco_Gb_Name("Alex");

oRecordInputParent.setZz_Tk_Person_Email(oNodeInputChild);

Zz_Tk_Person_Soap_RootRecord[] oRecordsetInputParent = { oRecordInputParent };
oNodeInputParent.setZz_Tk_Person_Soap_RootRecordSet(oRecordsetInputParent);

// Invoke the method that persists over the
PutpersonOutput oPutPersonServiceResult = oService.PUTPERSON(oNodeInputParent);

// Check the return type
double dReturn = oPutPersonServiceResult.get_return();
```

Ejemplo 3 (Modificar registro): Siguiendo el mismo ejemplo del código anterior, se trataría de recuperar los datos de la nueva persona, determinar su RID, y a continuación modificar alguno de sus campos.

Para obtener el RID de la nueva persona, bastaría con llamar al método de salida, recoger el objeto registro deseado, y ejecutar el método getM4AutoGeneratedRecordID

```

GetpersonOutput oGetPersonServiceResult = oService.GETPERSON(sNew_Std_Id_Person);

// Obtain the root block
Zz_Tk_Person_Soap_RootBlock oNodeParent =
oGetPersonServiceResult.getZz_Tk_Person_Soap_Root();

// Obtain the root recordset
Zz_Tk_Person_Soap_RootRecord[] oRecordsetParent =
oNodeParent.getZz_Tk_Person_Soap_RootRecordSet();

// Iterate each record
for (int i=0; i<oRecordsetParent.length; i++)
{
    if (sNew_Std_Id_Person.equals(oRecordsetParent[i].getStd_Id_Person()))
    {
        sTechIDRecord = oRecordsetParent[i].getM4AutoGeneratedRecordID();
    }
}
)

```

Para realizar la modificación del registro encontrado, sería necesario ejecutar el método `setM4AutoGeneratedRecordID` del objeto que representa el registro, pasándole el identificador recién obtenido. Esto debe hacerse en la misma sesión en la que se obtuvo el identificador y preferiblemente sobre el mismo objeto que vino de la transacción de salida.

```

// Change the family name "García" by "García-González"
oRecordParent.setM4AutoGeneratedRecordID(ai_sTechIDRecord);
oRecordParent.setStd_N_Family_Name_1("García-González");

Zz_Tk_Person_Soap_RootRecord[] oRecordsetInputParent = { oRecordParent };

oNodeParent.setZz_Tk_Person_Soap_RootRecordSet(oRecordsetParent);

// Invoke the method that persists
PutpersonOutput oPutPersonServiceResult = oService.PUTPERSON(oNodeParent);

double dReturn = oPutPersonServiceResult.get_return();

```

Ejemplo 4 (Borrar registro): En este caso, se debe recuperar el RID, y ejecutar el método `setM4AutoGeneratedRecordID` del objeto registro a borrar, en las mismas condiciones explicadas para la modificación, pero además es necesario fijar la marca de borrado, ejecutando el método `setM4AutoGeneratedToDelete` y fijando el valor a `true`.

•

```
// Build input block/recordset.  
oRecordParent.setM4AutoGeneratedToDelete (true);  
oRecordParent.setM4AutoGeneratedRecordID (ai_sTechIDRecord);  
  
Zz_Tk_Person_Soap_RootRecord[] oRecordsetParent = { oRecordParent } ;  
oNodeParent.setZz_Tk_Person_Soap_RootRecordSet (oRecordsetParent);  
  
// 4. Invoke the method that persists  
PutpersonOutput oPutPersonServiceResult = oService.PUTPERSON(oNodeParent);  
  
double dReturn = oPutPersonServiceResult.get_return();
```

Realizar verificación del servicio

Para probar las llamadas de consulta y modificación, se usará el código mostrado anteriormente, integrado con la autenticación, como se detalla en secciones posteriores.

Los siguientes puntos se deben tener en cuenta a la hora de poner en funcionamiento un servicio SOAP que utilice la persistencia a través de identificador tecnológico de registro.

Correspondencia del identificador con una sesión

En las llamadas al método de persistencia, debe tenerse en cuenta que el identificador de registro se corresponde con un Meta4Object que se queda levantado en el servidor. Por tanto para poder hacer una operación de persistencia, se debe reciclar en el fondo una instancia ya levantada, es decir no sólo se hace un GET para conseguir el número RID, sino que también se hace para cargar el canal y que se encuentre disponible en el servidor de aplicaciones cuando se ejecute el PUT.

En resumen, el usuario desarrollador de servicios SOAP hará una doble transacción, una de salida y otra de entrada. Esto se puede conseguir teniendo un método de salida y otro de entrada (GET y PUT), o bien, haciendo un solo método que ejecute una carga si el nodo no está cargado (llamada GET), y una llamada al método Persist_Tree si lo está (llamada PUT). Ambas transacciones deben tener de fondo la misma sesión, es decir, cualquier transacción de entrada tiene que ir precedida de una de salida inmediatamente antes y con la misma sesión.

Reportar errores de persistencia como excepciones SOAP

Cuando se inserta un registro que no se puede persistir correctamente, por ejemplo, porque se viola una clave primaria, o no se cumple una clave foránea, se esperaría que de algún modo el servicio respondiera dando una excepción

de SOAP.

Sin embargo, si se hace la prueba, de tener un método PUT que solamente haga un Persist_Tree, no resulta una excepción ante errores como estos:

log_message. El registro ya existe. No se puede insertar el registro "XML_ID=newfile.xml" de "STM_XML_FILES" porque ya existe. Debe cambiar el ID registro. Esto sucedió en "TST_SOAP_XMLFILE:_ROOT" de "TST_SOAP_XMLFILE".

log_message. Datos no válidos. No se puede insertar ni actualizar el registro "XML_ID=anotherfile.xml" de "STM_XML_FILES" porque los datos introducidos en "DTD_ID" no cumplen los requisitos de "STM_DTD_FILES". Debe cambiar el ID Registro. Esto sucedió en "TST_SOAP_XMLFILE:_ROOT" de "TST_SOAP_XMLFILE".

Para conseguirlo, se precisa modificar una de las propiedades avanzadas del método en cuestión. Esta propiedad es la "Transaccionalidad". Por defecto, un método recién creado "No soporta transacciones" ("Does not support transactions"). Eso quiere decir que si el Persist_Tree falla, la máquina virtual devuelve éxito en la ejecución del método PUT. Es preciso que incluya el método PUT en la transacción realizada por Persist_Tree para que el error que se produce al insertar en Base de Datos llegue hasta el cliente SOAP. En el diseñador de Meta4Objects, se debe marcar el método, en sus "Propiedades Avanzadas", como "Requiere transacción" ("Requires Transaction") para que este tipo de errores resulten en excepciones SOAP.

Borrado de jerarquías

En el caso de borrado de registros en nodos padre de otros nodos de una jerarquía exportada por SOAP, para que cuando se marca un registro padre como borrado, sus hijos también desaparezcan (en este ejemplo, para que si se borra una persona se borren todos sus correos electrónicos), en el conector se debe marcar como "Remove Child Nodes" = "True".

Es importante que cuando se borran registros no existan borrados en cascada con tablas que no existan, o de lo contrario la ejecución del borrado resultará en un error cuyo resultado es que no se podrán borrar los registros.

:

Tipo de método de negocio	Identificador del ejemplo	Propósito del método de negocio del ejemplo	Procedimiento para exportar la funcionalidad por SOAP
Método complejo con argumento de entrada (añadir, modificar o borrar registros)	putPerson	Permitir que se añada una persona completa a la tabla de personas, se modifiquen sus datos personales o se den de baja.	Sobre un Meta4Object apropiado, se define un método LN4, que simplemente persiste (Persist_Tree), sin argumentos simples, y añadir el nodo raíz como argumento complejo de entrada. ■ Añadir registros: se introducen en el argumento de entrada. ■ Modificar registros: se realiza operación de consulta, se obtiene el ID tecnológico del registro a modificar y se envía el argumento de entrada con las modificaciones pertinentes y el ID tecnológico que se desea cambiar. ■ Borrar registros: se realiza operación de consulta, se obtiene el ID tecnológico del registro a borrar y se envía el argumento de entrada con el ID tecnológico que se desea cambiar, y la marca de borrado.
Método complejo con argumento de salida	getPerson	Dado un identificador de persona, retomar toda la información necesaria sobre la misma.	Crear un Meta4Object con su nodo raíz asociado a una estructura de nodo sobre la tabla de personas (STD_PERSON), crear un nuevo método LN4. En la regla asociada, cargar el nodo (Load_Prg). Para que sólo se carguen en el nodo los registros pertinentes, establecer previamente una sentencia (SYS_SENTENCE) adecuada. Al publicar este método, indicar el Meta4Object, nodo e ítem del método LN4, el nombre público del método de negocio, y que existe un argumento complejo de salida, que es el nodo raíz.

Tabla: Cuadro resumen del ejemplo de persistencia

Servicio sobre un Meta4Object con soporte a ficheros

Se dispone de una tabla con ficheros (tabla física M4RTM_XML_FILES, tabla lógica STM_XML_FILES), con un campo de tipo BLOB (XML_FILE), en la que se desea insertar un fichero, para recuperarlo a continuación a modo de comprobación. Para ello, se creará un Meta4Object sobre esta tabla, llamado TST_SOAP_XMLFILES. En este Meta4Object se un método PUTFILE, con una única llamada a la persistencia. El método se publicará como método de

negocio, con un argumento de entrada/salida. En esta sección sólo se discutirá cómo elaborar el código cliente.

Elaborar el código cliente

Para elaborar el código cliente se necesitará un entorno Java, puesto que en el WSDL aparece el tipo `apachesoap:DataHandler`, que no es compatible con otras plataformas.

`DataHandler` es un objeto del Activation Framework de Java que contiene un fichero, junto con un campo que describe su contenido (tipo MIME).

A continuación se muestra el código Java que se necesitaría para añadir un registro con un fichero a través del Meta4Object `TST_SOAP_XMLFILES` en un entorno Java/Axis.

```
String tempDir = "c:\\temp\\fileput\\";
String filename = "newfile.xml";

// =====
// 1. Instantiate a proxy factory
Tst_Soap_XmlfileServiceService factory = new Tst_Soap_XmlfileServiceServiceLocator();
// 2. Instantiate a proxy
Tst_Soap_XmlfileService oSoapFileService = factory.getTST_SOAP_XMLFILE();

/* Insert here the authentication code */

// 3a. Load the test file into a DataHandler (has the file and the content-type)
// =====
FileDataSource fds = new FileDataSource(tempDir + filename);
DataHandler attachment = new DataHandler(fds);

// 3b. Build input block/recordset.
Tst_Soap_Xmlfile_RootBlock oNodeInput = new Tst_Soap_Xmlfile_RootBlock();
Tst_Soap_Xmlfile_RootRecord oRecordInput = new Tst_Soap_Xmlfile_RootRecord();
oRecordInput.setXml_Id("ParametersFile-1224.xml"); // PK
oRecordInput.setDtd_Id("Parameters.dtd"); // FK to DTDs table
oRecordInput.setXml_Description("test");
oRecordInput.setXml_Name("File inserted with a SOAP service");
oRecordInput.setXml_File(attachment);

Tst_Soap_Xmlfile_RootRecord[] oRecordsetInput = { oRecordInput };
oNodeInput.setTst_Soap_Xmlfile_RootRecordSet ( oRecordsetInput );

// 4. Invoke the method that persists
PutfileOutput oNodeOutput = oSoapFileService.PUTFILE ( oNodeInput );

// 5. Obtain the return code. 0 is ok.
double dReturn = oNodeOutput.get_return();
```

Si el código se ha ejecutado con éxito (es decir, el retorno es igual a 0), es posible recuperar en forma de fichero el resultado, a partir del objeto `oNodeOutput`, con un código similar al que se ofrece en el ejemplo.

}

```
Tst_Soap_Xmlfile_RootBlock oBlockOutput = oNodeOutput.getTst_Soap_Xmlfile_Root();
Tst_Soap_Xmlfile_RootRecord[] oRecordsetOutput =
bloque.getTst_Soap_Xmlfile_RootRecordSet();

int iSize = oRecordsetOutput.length;
DataHandler attachmentEcho = null;

for (int i=0; i<iSize; i++)
{
attachmentEcho = oRecordsetOutput[i].getXml_File();

ManagedMemoryDataSource mmnds =
(ManagedMemoryDataSource)attachmentEcho.getDataSource();
if (mmnds != null)
{
InputStream is = mmnds.getInputStream();
File attachedFile = null;

attachedFile = new File(tempDir + "echo" + filename);

BufferedOutputStream bos =
new BufferedOutputStream(new FileOutputStream(attachedFile));
int size = 0;
byte[] buf = new byte[1024*8];
while ((size = is.read(buf)) > -1) {
bos.write(buf, 0, size);
}
bos.flush();
bos.close();

System.out.println("File was downloaded to: " +
"" + attachedFile.toString());

mmnds.delete();
}
}
```

:

Tipo de método de negocio	Identificador del ejemplo	Propósito del método de negocio del ejemplo	Procedimiento para exportar la funcionalidad por SOAP
Método complejo con ficheros de entrada	putFiles	Permitir que se añada un fichero concreto, o se modifique o borre	Crear un Meta4Object con su nodo raíz con una estructura de nodo sobre la tabla en cuestión, un método con una regla que no tiene más que una llamada a la persistencia, y al publicar este método, indicar que tiene un argumento de entrada, que es el nodo raíz.

Tipo de método de negocio	Identificador del ejemplo	Propósito del método de negocio del ejemplo	Procedimiento para exportar la funcionalidad por SOAP
Método complejo con ficheros de salida	getFiles	Permitir que se obtengan todos los ficheros de una tabla que contiene items de tipo blob	Crear un Meta4Object con su nodo raíz con una estructura de nodo sobre la tabla en cuestión, un método con una regla que no tiene más que una carga del nodo raíz, y al publicar este método, indicar que tiene un argumento de entrada, que es el nodo raíz.

Tabla: Cuadro resumen del ejemplo de ficheros

Autenticación de los servicios SOAP

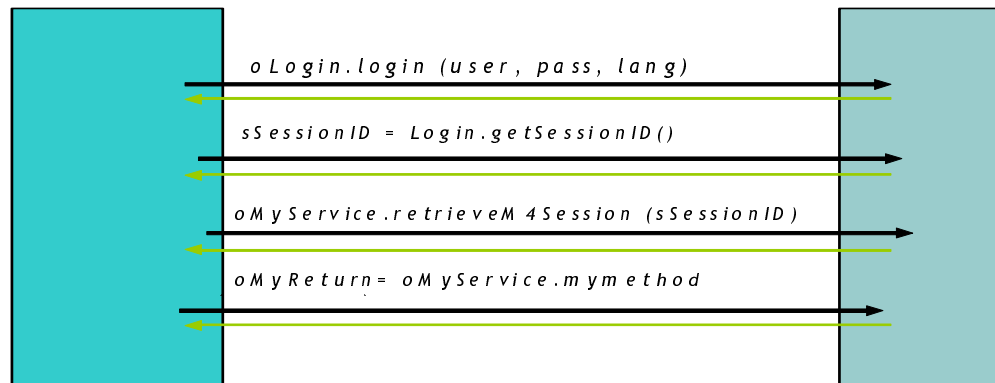
En los ejemplos que hemos explorado anteriormente, sería necesario haber establecido una sesión para poder proceder, puesto que sin una sesión para SOAP, Meta4 no permite el acceso a sus datos. Existen distintas variantes de código cliente para establecer una autenticación en un servicio de Meta4.

La más sencilla (autenticación clásica) es la que se establece mediante la ejecución de un método de un servicio auxiliar, que proporciona una sesión y un identificador de la misma, que permite a otros servicios ejecutarse con la seguridad apropiada.

Autenticación clásica (ejecución de servicio de login)

Existe un servicio de Meta4 llamado Login, con un método login, al que si se proporciona usuario, contraseña e idioma, genera sesiones de Meta4 para SOAP. Dicho método devuelve un identificador único de la sesión. Existe otro método logout en el mismo servicio para invalidar y desconectar dichas sesiones.

El modo clásico de autenticarse para llamar los servicios SOAP de Meta4 es realizar una secuencia de tres llamadas. Primero, se llama al método de login del servicio Login, para obtener una sesión y su identificador. Después, se llama al método retrieveM4Session del servicio (todos los servicios tienen este método). Y por último se efectúa la llamada. El estado de esta última llamada con respecto a la ejecución de retrieveM4Session se controla a través de cookies HTTP.



Codificación

Para elaborar el código cliente de llamada a la autenticación, es necesario desplegar el servicio Login (esto se puede llevar a cabo mediante las páginas de administración de servicios SOAP de Meta4). A continuación se obtendrá su WSDL (<http://servidor:puerto/services/Login?wsdl>) y se crearán las clases Proxy del mismo. Con esas clases Proxy, se puede escribir un código como el siguiente para obtener un identificador de sesión (ejemplo en Java):

```

// 0. Prepare authentication calling the login
// =====
// A1. Instantiate a proxy factory
LoginService sfactory = new LoginServiceLocator();
// A2. Instantiate a proxy
Login login = sfactory.getLogin();
// A3. Invoke the web service's getList() method
M4LoginOutput loutput = login.login(sUser, sPass, sLang);
// A4. Get the session id
String sessionid = loutput.getSessionID();
  
```

Si el identificador de sesión se obtiene con éxito, antes de llamar al método `retrieveM4Session` del servicio, será necesario fijar las propiedades necesarias para que las llamadas que lo necesiten puedan encadenarse a través de cookies. Este código es dependiente del entorno de desarrollo:

C# (Microsoft VS .Net)

Para poder invocar varios servicios desde clientes generados por el asistente de VS .Net (archivo `Reference.cs`), hay que introducir en el constructor del objeto servicio (`*ServiceService`) la línea:

```
CookieContainer = new System.Net.CookieContainer();
```

En caso contrario aparece una excepción con el texto: "User has not a valid M4 Session".

"Java (Eclipse/Axis)

Con un cliente Axis, basta con poner a true la propiedad SESSION_MAINTAIN_PROPERTY del objeto Stub que se utiliza para ejecutar cada llamada al servicio.

```
// Ensure the service is properly authenticated
// =====
javax.xml.rpc.Stub oStub = (javax.xml.rpc.Stub) oSoapFileService;

oStub._setProperty(Stub.SESSION_MAINTAIN_PROPERTY, new Boolean(true));
```

Por último, se lanzará la llamada al método retrieveM4Session, pasándole el identificador de sesión y controlando el retorno mediante un entero.

```
int iSuccess = oSoapFileService.retrieveM4Session ( sessionid );
```

Si en la llamada al método login no se puede obtener un identificador de sesión porque se produce un error (contraseña incorrecta, servidor de aplicaciones no disponible) se recogerá una excepción en el código de invocación, de la que se podrá inferir la causa.

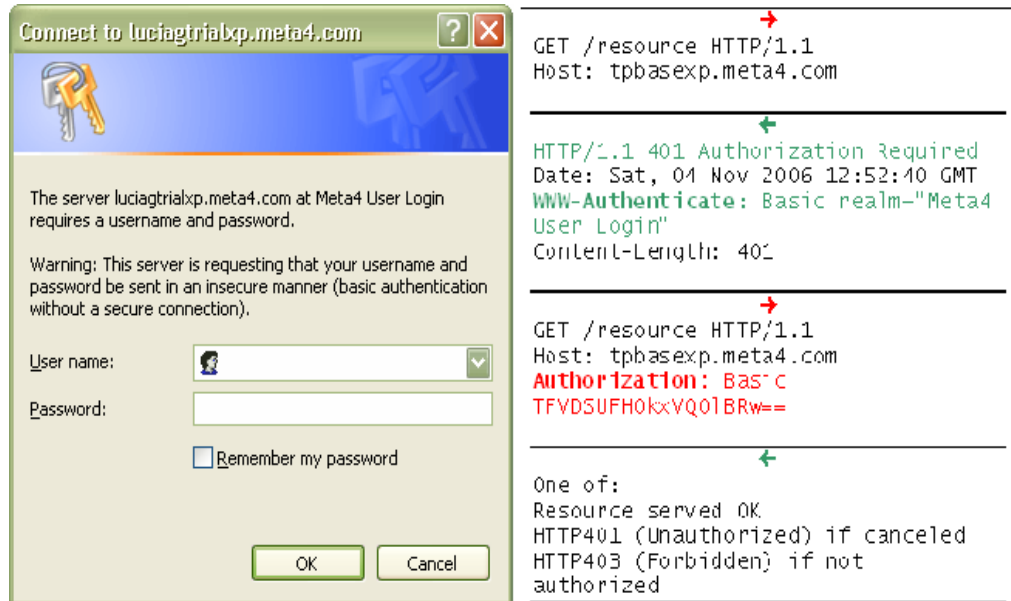
```
try
{
    /* login call + method call */
}
catch (Exception e)
{
    e.printStackTrace();
    AxisFault mse = M4SoapException.makeFault(e);
    String sFaultCode = mse.getFaultCode().getLocalPart();
    String sFaultString = mse.getFaultString();

    /* print the error strings or send them to an error device */
}
```

Autenticación básica HTTP

Para aquellos clientes que no deseen utilizar la autenticación clásica de Meta4, existe la posibilidad de utilizar las posibilidades de Autenticación Básica del protocolo HTTP (según RFC 2617).

Esta autenticación asume que al acceder a un recurso "protegido", el usuario se identifica mediante usuario y contraseña, datos que viajan codificados en una cabecera HTTP, que el servidor de SOAP es capaz de decodificar para conceder una sesión al usuario.



Cuando una aplicación web se configura para usar autenticación básica (BASIC en el elemento auth-method de web.xml), el contenedor web usa el esquema desafío/respuesta para levantar una ventana del navegador web, con la que preguntar un usuario/contraseña si se solicita algo protegido.

Para implementar este tipo de autenticación para los servicios SOAP, por lo tanto:

El servicio debe ser "protegido", es decir, incluido en la lista de los recursos cuyo acceso iniciará un "Desafío" HTTP (lanzamiento de cabecera WWW-Authenticate con el dominio o Realm de autenticación).

El cliente debe proporcionar un usuario/contraseña (en la cabecera Authorization) que coincidan exactamente con los que tiene ese usuario en el repositorio de Meta4.

Para proteger un recurso J2EE con autenticación básica, lo que se hace es definir restricciones de seguridad en el descriptor de la aplicación web. Un ejemplo de configuración de esas restricciones para "proteger" un recurso como por ejemplo el servicio ZZ_TK_PERSON_SOAP sería el siguiente fragmento de web.xml.

```
<security-constraint>
  <web-resource-collection>
    <web-resource-name>Protected Services</web-resource-name>
    <url-pattern>/services/ZZ_TK_PERSON_SOAP</url-pattern>
    <http-method>GET</http-method>
    <http-method>POST</http-method>
  </web-resource-collection>
  <auth-constraint>
    <role-name>serviceuser</role-name>
  </auth-constraint>
</security-constraint>
<login-config>
  <auth-method>BASIC</auth-method>
  <realm-name>Meta4 User Realm (BASIC)</realm-name>
</login-config>
<security-role>
  <role-name>serviceuser</role-name>
</security-role>
```

Es necesario llevar a cabo un paso más, que es configurar el servidor web para que acepte los usuarios introducidos. Para pruebas, será suficiente utilizar el almacén de usuarios por defecto del contenedor web, que suele ser un fichero XML con los usuarios y contraseñas. Esto no quiere decir que sea estrictamente necesario almacenar las contraseñas en un fichero XML del contenedor web, sino que de esta manera podemos probar la autenticación sin definir más integración con los usuarios Meta4.

Para configurar el almacén por defecto, por ejemplo, en el caso de JRun, en el fichero <jrunroot>\servers\<serverid>\SERVER-INF\jrun-users.xml se pueden consignar todos los usuarios que se necesita que utilicen este tipo de autenticación. Si se desea que sus contraseñas estén codificadas, habría que encriptarlo, p.e. (<jrun-root>\lib>java -classpath ./jrun.jar jrun.security.JRunCrypterImpl M4ADM). Se debe también introducir el rol que definimos para los que pueden acceder. En Tomcat es el fichero CATALINA_BASE/conf/tomcat-users.xml.

```

<jrun-users>
  <encryption>true</encryption>
  <user>
    <user-name>jmsuser</user-name>
    <password>CwN66L4zEXtDU</password>
  </user>
  <user>
    <user-name>M4ADM</user-name>
    <password>39rUbFJO0KoQ.</password>
  </user>
  <role>
    <role-name>serviceuser</role-name><description/>
    <user-name>M4ADM</user-name>
  </role>
</jrun-users>

```

Con esta configuración, podremos acceder al servicio protegido sin necesidad de llamar a ningún método previo. Es recomendable configurar SSL en el lado servidor, para evitar la interceptación de contraseñas codificadas.

Codificación

Si el cliente es programático, existe un modo de proporcionar usuario/contraseña sin esperar al desafío.

Java (Eclipse/Axis)

Con un cliente Axis, se consigue la autenticación básica fijando el usuario y la contraseña en las propiedades del objeto Stub `USERNAME_PROPERTY` y `PASSWORD_PROPERTY` respectivamente.

```

javax.xml.rpc.Stub oStub = (javax.xml.rpc.Stub) oSoapFileService;

oStub._setProperty(Stub.USERNAME_PROPERTY, sUser);
oStub._setProperty(Stub.PASSWORD_PROPERTY, sPass);

```

Autenticación basada en WS-Security

Actualmente, la tendencia de los servicios SOAP es seguir el estándar WS-Security (WSS) en cuestiones de seguridad. Para la autenticación, se incluye información en las cabeceras SOAP en lugar de hacerlo en mensajes

adicionales (autenticación clásica) o en las cabeceras de transporte HTTP. Los servicios SOAP de Meta4 implementan "Username Token Profile 1.0" con contraseñas en modo texto.

El mensaje SOAP con soporte a dicha autenticación presenta un aspecto como el siguiente:

:

```
<?xml version="1.0" encoding="UTF-8" ?>
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope"
  xmlns:xsd="http://www.w3.org/2001/XMLSchema"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
  <soapenv:Header>
    <wsse:Security xmlns:wsse="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-secext-1.0.xsd" soapenv:mustUnderstand="1">
      <wsse:UsernameToken xmlns:wsu="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd" wsu:Id="UsernameToken-3138643">
        <wsse:Username>M4ADM</wsse:Username>
        <wsse:Password Type="http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd#PasswordText">M4ADM</wsse:Password>
      </wsse:UsernameToken>
    </wsse:Security>
  </soapenv:Header>

  <soapenv:Body>
    <!-- Body goes here -->
  </soapenv:Body>
</soapenv:Envelope>
```

Para que los servicios SOAP de Meta4 sean capaces de recoger el usuario/contraseña de un mensaje de este tipo, sin requerir programación por parte del usuario, solamente es preciso definir que se desea seguridad tipo WS-Security en la presentación de publicación de los métodos de negocio. Esto introduce en el descriptor de servicio (WSD) una referencia a una clase que entiende estas cabeceras y las envía a validar al servidor de aplicaciones.

Cuando en la presentación de publicación de servicio se marca "ID Seguridad Web Services" con valor 1, los servicios autogenerados por Meta4 llevan en su descriptor de despliegue la referencia a un manejador de autenticación como se ve a continuación.

```

<requestFlow>
  <handler type="java:org.apache.ws.axis.security.WSDoAllF
    <parameter name="passwordCallbackClass"
      value="com.meta4.soapservices.session.auth.
        M4SoapAuthenticatorUsernameToken"/>
    <parameter name="action" value="UsernameToken"/>
  </handler>
</requestFlow>

```

Codificación

Crear un cliente que soporte WS-Security implica varios pasos. En primer lugar es preciso crear un descriptor del cliente, para indicar qué manejador se encargará de construir la petición. Con AXIS se distribuye un manejador, llamado WSDoAllSender, que nos será útil para realizar una prueba con las contraseñas en modo texto.

Llamaremos a este fichero client-config.wsdd.

```

<deployment xmlns="http://xml.apache.org/axis/wsdd/"
  xmlns:java="http://xml.apache.org/axis/wsdd/providers/java">
  <transport name="http" pivot="java:org.apache.axis.transport.http.HTTPSender" />
</globalConfiguration>
<requestFlow>
  <handler type="java:org.apache.ws.axis.security.WSDoAllSender">
    <parameter name="passwordType" value="PasswordText" />
  </handler>
</requestFlow>
</globalConfiguration>
</deployment>

```

Este manejador cliente no permite introducir la contraseña en claro, sino que requiere desarrollar una clase que implementa una interfaz, como se indica a continuación.

```

/*
 * Created on 01/08/2007
 */
package com.meta4.servicestubs;
import javax.security.auth.callback.Callback;
import javax.security.auth.callback.CallbackHandler;
import javax.security.auth.callback.UnsupportedCallbackException;

import org.apache.ws.security.WSPasswordCallback;

import java.io.IOException;

public class PasswordProvider implements CallbackHandler {

    /**
     * (non-Javadoc)
     * @see javax.security.auth.callback.CallbackHandler#handle(javax.security.auth.callback.Callback[])
     */
    /**
     * Method handle
     *
     * @param callbacks
     * @throws IOException
     * @throws UnsupportedCallbackException
     */
    public void handle(Callback[] callbacks)
        throws IOException, UnsupportedCallbackException {

        for (int i = 0; i < callbacks.length; i++) {
            if (callbacks[i] instanceof WSPasswordCallback) {
                WSPasswordCallback pc = (WSPasswordCallback) callbacks[i];

                /*
                 * here call a function/method to lookup the password for
                 * the given identifier (e.g. a user name or keystore alias)
                 * e.g.:
                pc.setPassword(passStore.getPassword(pc.getIdentifer()))
                */

                if (pc.getIdentifer().equals("M4ADM"))
                {
                    pc.setPassword("M4ADM");
                }

            } else {
                throw new UnsupportedCallbackException(callbacks[i],
                    "Unrecognized Callback");
            }
        }
    }
}

```

Una vez descrito el manejador en el WSDD cliente, e implementada la clase que recupera las contraseñas en modo texto, se puede escribir el código cliente que llamará al servicio. En este ejemplo se llamará al servicio ZZ_APPUSER_QUERY.

```
// Define the WSDD configuration on an invocation-by-invocation basis within the Client code.
// For example, the Client code may contain the following invocations:

EngineConfiguration config = new FileProvider("D:/files/client_deploy.wsdd");

// Instantiate a proxy factory
Zz_Appuser_QueryServiceService factory = new Zz_Appuser_QueryServiceServiceLocator(config);
// Instantiate a proxy
Zz_Appuser_QueryService oService = factory.getZZ_APPUSER_QUERY();

// Instantiate a proxy factory
oStub._setProperty(WSHandlerConstants.ACTION, WSHandlerConstants.USERNAME_TOKEN);
oStub._setProperty(WSHandlerConstants.USER, "M4ADM");
oStub._setProperty(WSHandlerConstants.PW_CALLBACK_CLASS,
"com.meta4.servicestubs.PasswordProvider");

PasswordProvider oCallback = new PasswordProvider();
oStub._setProperty(WSHandlerConstants.PW_CALLBACK_REF, oCallback);
```

Autenticación integrada con seguridad J2EE

Si un usuario se ha autenticado según la especificación J2EE en el contenedor web en el que corre el servicio de Meta4, el API de J2EE de seguridad (getRemoteUser, getUserPrincipal) será capaz de obtener su nombre.

En ese caso, y si se ha generado desde la presentación correspondiente el almacén de claves de Meta4, se intentará realizar una autenticación pasando en la contraseña una credencial encriptada utilizando el API de generación de credenciales que ya se utiliza en el SSO con portales.

Se leerán los valores del fichero y de la clave desde propiedades de Java (-Dpropiedad) y las propiedades se llaman meta4.filepath y meta4.keypass. Si el fichero almacén de claves no existe, no se intenta nada. Asimismo, si no se ha podido obtener el nombre con el API J2EE, no se hará nada. Si se hace un viaje al servidor de aplicaciones y falla el login, no se hará nada, pero finalmente, y como es el último mecanismo de autenticación actualmente implementado, acabará saliendo un error sobre la no autenticación del usuario.

En resumen, para configurar este mecanismo, además de estar implementada en el contenedor la seguridad web J2EE, deben realizarse las siguientes operaciones:

- **Repositorio de Meta4:** Crear el fichero ca.p12 mediante presentación a tal efecto incluida (ver documentación del Single Sign On con Portales). Este fichero debe situarse en una ruta visible desde el contenedor web, por ejemplo, <raiz>/WEB-INF/classes/properties/ca.p12

-
- **Fichero <jrun-root>\bin\jvm.config:** Introducir las propiedades relativas a dicho fichero en el campo java.args. Se debe reiniciar el contenedor web para que dichas propiedades se carguen.

```
java.args=-Xmx128M -Xms32m -Dmeta4.filepath=<raiz>/WEB-INF/classes/properties/ca.p12 -  
Dmeta4.keypass=changeit
```

Los clientes SOAP de Meta4 en detalle

5

Acerca de este capítulo

En este capítulo se profundiza, mediante ejemplos, en el desarrollo, seguridad y ciclo de vida de los clientes SOAP de Meta4.

Cliente de un servicio SOAP de autenticación clásica de Meta4 (ejecución del servicio de login)

Existe un servicio de Meta4 llamado Login, con un método login, al que si se proporciona usuario, contraseña e idioma, genera sesiones de Meta4 para SOAP. Dicho método devuelve un identificador único de la sesión. Existe otro método logout en el mismo servicio para invalidar y desconectar dichas sesiones.

El modo clásico de autenticarse para llamar los servicios SOAP de Meta4 es realizar una llamada al método de login del servicio Login, para obtener una sesión y su identificador.

A continuación se mostrará un ejemplo para poder ejecutar el servicio de login desde un método LN4 de un Meta4Object. El método sigue estos pasos:

1. Realiza una llamada al servicio Login ("http://localhost:8080/services/Login")
2. Recupera el identificador de la sesión.
3. Se desloga.

EJEMPLO

```
iRet = M4_ERROR
```

```

// Load Context
iRet = SCH_JAVA_API.LOAD_CONTEXT("LOGIN")
if (iRet <> M4_SUCCESS) then
    return M4_ERROR
endif

oUrl = SCH_JAVA_API.CREATE_OBJECT("java.net.URL", iRet, "http://
localhost:8080/services/Login")
if (iRet <> M4_SUCCESS) then
    return M4_ERROR
endif

oFactory =
SCH_JAVA_API.CREATE_OBJECT("com.meta4.schemas.LoginServiceServiceLoca
tor", iRet)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    return M4_ERROR
endif

oService = SCH_JAVA_API.INVOKE_METHOD(oFactory, "getLogin", iRet,
oUrl)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    return M4_ERROR
endif

oBoolean = SCH_JAVA_API.CREATE_OBJECT("java.lang.Boolean", iRet,
"true")
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    return M4_ERROR
endif

// Login
sUser = "M4ADM"
sPassword = "M4ADM"
sLang = "2"
oLoginOutput = SCH_JAVA_API.INVOKE_METHOD(oService, "login", iRet,
sUser, sPassword, sLang)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    return M4_ERROR
endif

sSessionId = SCH_JAVA_API.INVOKE_METHOD(oLoginOutput, "getSessionID",
iRet)
if (iRet <> M4_SUCCESS) then

```

```

        SCH_JAVA_API.DESTROY_OBJECT(oUrl)
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
        return M4_ERROR
    endif

    // Logout
    SCH_JAVA_API.INVOKE_METHOD(oService, "logout", iRet)
    if (iRet <> M4_SUCCESS) then
        SCH_JAVA_API.DESTROY_OBJECT(oUrl)
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
        return M4_ERROR
    endif

    // Destroy objects.
    iRet = SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    iRet = SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    iRet = SCH_JAVA_API.DESTROY_OBJECT(oBoolean)

    return M4_SUCCESS

```

Cliente de un servicio SOAP de Meta4 (con operaciones de consulta)

En esta sección se muestra un ejemplo de código de cómo ejecutar un cliente de un servicio SOAP desde un Meta4Object para realizar operaciones de consulta.

Este método sigue los pasos que se indican a continuación:

1. Realiza una llamada al servicio Login ("http://localhost:8080/services/Login")
2. Llama al método de consulta de información de las personas pasándole como argumento el ID persona M10028 y consulta el resultado.
3. Recupera la información de la persona M10028, leyendo el nombre y algunos de sus datos personales.
4. Recupera la información de sus correos electrónicos.
5. Se desloga.

EJEMPLO

```
iRet = M4_ERROR
```

```

sStdIdPerson = "M10028"

'-----
-
'Login
'-----
-

iRet = SCH_JAVA_API.LOAD_CONTEXT("LOGIN")
If iRet <> M4_SUCCESS Then
    Return(M4_ERROR)
EndIf

oFactory =
SCH_JAVA_API.CREATE_OBJECT("com.meta4.schemas.LoginServiceServiceLoca
tor", iRet)
If iRet <> M4_SUCCESS Then
    Return(M4_ERROR)
EndIf

oUrl = SCH_JAVA_API.CREATE_OBJECT("java.net.URL", iRet, "http://
localhost:8080/services/Login")
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    Return(M4_ERROR)
EndIf

oServiceLogin = SCH_JAVA_API.INVOKE_METHOD(oFactory, "getLogin",
iRet, oUrl)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    Return(M4_ERROR)
EndIf

ologOutput = SCH_JAVA_API.INVOKE_METHOD(oServiceLogin, "login", iRet,
"M4ADM", "M4ADM", "2")
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    Return(M4_ERROR)
EndIf

sSessionId = SCH_JAVA_API.INVOKE_METHOD(ologOutput, "getSessionID",
iRet)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    Return(M4_ERROR)
EndIf

oBoolean = SCH_JAVA_API.CREATE_OBJECT("java.lang.Boolean", iRet,
"true")

```

```

If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    Return(M4_ERROR)
EndIf

SCH_JAVA_API.INVOKE_METHOD(oServiceLogin, "_setProperty", iRet,
"javax.xml.rpc.session.maintain", oBoolean)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    Return(M4_ERROR)
EndIf

SCH_JAVA_API.INVOKE_METHOD(oServiceLogin, "retrieveM4Session", iRet,
sSessionId)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    Return(M4_ERROR)
EndIf

'Destroy objects.
SCH_JAVA_API.DESTROY_OBJECT(oFactory)
SCH_JAVA_API.DESTROY_OBJECT(oUrl)
SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
'-----
-
'-----
-

// Load context.
iRet = SCH_JAVA_API.LOAD_CONTEXT("ZZ_TK_PERSON_LITERAL_SOAP")
if (iRet <> M4_SUCCESS) then
    return M4_ERROR
endif

oZz_Tk_Person_Literal_Soap_Address =
SCH_JAVA_API.CREATE_OBJECT("java.net.URL", iRet, "http://
localhost:8080/services/ZZ_TK_PERSON_LITERAL_SOAP")
if (iRet <> M4_SUCCESS) then
    return M4_ERROR
endif

oFactory =
SCH_JAVA_API.CREATE_OBJECT("com.meta4.schemas.Zz_Tk_Person_Literal_So
apServiceServiceLocator", iRet)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oZz_Tk_Person_Literal_Soap_Address)
    return M4_ERROR

```

```

endif

oService = SCH_JAVA_API.INVOKE_METHOD(oFactory,
"getZZ_TK_PERSON_LITERAL_SOAP", iRet,
oZZ_Tk_Person_Literal_Soap_Address)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oZZ_Tk_Person_Literal_Soap_Address)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    return M4_ERROR
endif

oBoolean = SCH_JAVA_API.CREATE_OBJECT("java.lang.Boolean", iRet,
"true")
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oZZ_Tk_Person_Literal_Soap_Address)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    return M4_ERROR
endif

SCH_JAVA_API.INVOKE_METHOD(oService, "_setProperty", iRet,
"javax.xml.rpc.session.maintain", oBoolean)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oZZ_Tk_Person_Literal_Soap_Address)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    return M4_ERROR
endif

SCH_JAVA_API.INVOKE_METHOD(oService, "retrieveM4Session", iRet,
sSessionId)
if (iRet <> M4_SUCCESS) then
    return M4_ERROR
endif

oGetpersonDoclitOutput = SCH_JAVA_API.INVOKE_METHOD(oService,
"GETPERSON_DOCLIT", iRet, sStdIdPerson)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oZZ_Tk_Person_Literal_Soap_Address)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    return M4_ERROR
endif

dResult = SCH_JAVA_API.INVOKE_METHOD(oGetpersonDoclitOutput,
"get_return", iRet)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(oZZ_Tk_Person_Literal_Soap_Address)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    return M4_ERROR
endif

```

```

ozzTkPersonSoapRootBlock =
SCH_JAVA_API.INVOKE_METHOD(oGetpersonDoclitOutput,
"getZz_Tk_Person_Soap_Root", iRet)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    return M4_ERROR
endif

ozzTkPersonSoapRootRecordArray =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonSoapRootBlock,
"getZz_Tk_Person_Soap_RootRecordSet", iRet)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    return M4_ERROR
endif

iArrayLen =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonSoapRootRecordArray, "size",
iRet)
if (iRet <> M4_SUCCESS) then
    SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    return M4_ERROR
endif

for i = 0 To iArrayLen - 1
    ozzTkPersonSoapRootRecord =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonSoapRootRecordArray, "get",
iRet, i)
    if (iRet <> M4_SUCCESS) then
        SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
        return M4_ERROR
    endif

    sFirstName =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonSoapRootRecord,
"getStd_N_First_Name", iRet)
    if (iRet <> M4_SUCCESS) then
        SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
        return M4_ERROR
    endif
endif

```

```

        sFamilyName =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonSoapRootRecord,
"getStd_N_Family_Name_1", iRet)
        if (iRet <> M4_SUCCESS) then
            SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
            SCH_JAVA_API.DESTROY_OBJECT(oFactory)
            SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
return M4_ERROR
        endif

        ozzTkPersonEmailBlock =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonSoapRootRecord,
"getZz_Tk_Person_Email", iRet)
        if (iRet <> M4_SUCCESS) then
SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
            SCH_JAVA_API.DESTROY_OBJECT(oFactory)
            SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
            return M4_ERROR
        endif

        ozzTkPersonEmailRecordArray =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonEmailBlock,
"getZz_Tk_Person_EmailRecordSet", iRet)
        if (iRet <> M4_SUCCESS) then
            SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
            SCH_JAVA_API.DESTROY_OBJECT(oFactory)
            SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
            return M4_ERROR
        endif

        if (IsNull(ozzTkPersonEmailRecordArray) = M4_FALSE And
ozzTkPersonEmailRecordArray <> 0) then
            iEmailArrayLen =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonEmailRecordArray, "size", iRet)
            if (iRet <> M4_SUCCESS) then
SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
                SCH_JAVA_API.DESTROY_OBJECT(oFactory)
                SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
                return M4_ERROR
            endif

            for j = 0 To iEmailArrayLen - 1
                ozzTkPersonEmailRecord =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonEmailRecordArray, "get", iRet,
j)
                if (iRet <> M4_SUCCESS) then

SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
                    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
                    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
return M4_ERROR
                endif
            
```

```

        sEmail =
SCH_JAVA_API.INVOKE_METHOD(ozzTkPersonEmailRecord, "getStd_Email",
iRet)
        if (iRet <> M4_SUCCESS) then
SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
return M4_ERROR
        endif
        next

    endif

next

'-----
-
'-----
-
ologOutput = SCH_JAVA_API.INVOKE_METHOD(oServiceLogin, "logout",
iRet)

// Release objects.
SCH_JAVA_API.DESTROY_OBJECT(ozz_Tk_Person_Literal_Soap_Address)
SCH_JAVA_API.DESTROY_OBJECT(oFactory)
SCH_JAVA_API.DESTROY_OBJECT(oBoolean)

return M4_SUCCESS

```

Cliente de un servicio SOAP de Meta4 con un fichero de entrada

En esta sección se muestra un ejemplo de código de cómo ejecutar un cliente de un servicio SOAP con ficheros de entrada desde un Meta4Object.

Este método sigue los pasos que se indican a continuación:

1. Crea una credencial para el usuario "myuser". Este identificador de usuario debe corresponder a un mapeo en el dominio de autenticación en el que se haya definido el fichero de extensión .p12, como se indica en el documento sobre Single Sign-On Web.
2. Realiza una llamada al servicio Login ("http://localhost:8080/services/Login").
3. Construye un objeto DataHandler con el fichero de entrada.

4. Construye el argumento complejo de entrada asociándole el objeto DataHandler.
5. Llama al método y consulta el resultado.
6. Se desloga.

EJEMPLO

```

iRet = -1

'-----
-
'Create credentials
'-----
-
oM4Credential =
SCH_JAVA_API.CREATE_OBJECT("com.meta4.common.m4cred.M4Credential",
iRet)
If iRet <> M4_SUCCESS Then
    Return(M4_ERROR)
EndIf

sCredential = SCH_JAVA_API.INVOKE_METHOD(oM4Credential,
"GenerateM4Credential", iRet, "C:\p12\mycredentials.p12", "mypassw",
"myuser", "3", 60, 600)

'Destroy object
SCH_JAVA_API.DESTROY_OBJECT(oM4Credential)

If iRet <> M4_SUCCESS Then
    Return(M4_ERROR)
EndIf

'-----
-
>Login
'-----
-
iRet = SCH_JAVA_API.LOAD_CONTEXT("LOGIN")
If iRet <> M4_SUCCESS Then
    Return(M4_ERROR)
EndIf

oFactory =
SCH_JAVA_API.CREATE_OBJECT("com.meta4.schemas.LoginServiceServiceLoca
tor", iRet)
If iRet <> M4_SUCCESS Then
    Return(M4_ERROR)
EndIf

oUrl = SCH_JAVA_API.CREATE_OBJECT("java.net.URL", iRet, "http://
localhost:8080/services/Login")

```

```

If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    Return(M4_ERROR)
EndIf

oServiceLogin = SCH_JAVA_API.INVOKE_METHOD(oFactory, "getLogin",
iRet, oUrl)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    Return(M4_ERROR)
EndIf

ologOutput = SCH_JAVA_API.INVOKE_METHOD(oServiceLogin, "login", iRet,
"myuser", sCredential, "3")
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    Return(M4_ERROR)
EndIf

sSessionId = SCH_JAVA_API.INVOKE_METHOD(ologOutput, "getSessionID",
iRet)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    Return(M4_ERROR)
EndIf

oBoolean = SCH_JAVA_API.CREATE_OBJECT("java.lang.Boolean", iRet,
"true")
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    Return(M4_ERROR)
EndIf

'Maintain object active for logout
SCH_JAVA_API.INVOKE_METHOD(oServiceLogin, "_setProperty", iRet,
"javax.xml.rpc.session.maintain", oBoolean)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
    Return(M4_ERROR)
EndIf

SCH_JAVA_API.INVOKE_METHOD(oServiceLogin, "retrieveM4Session", iRet,
sSessionId)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)

```

```

        SCH_JAVA_API.DESTROY_OBJECT(oBoolean)
        Return(M4_ERROR)
    EndIf

    'Destroy objects.
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)

    '-----
    -
    '-----
    -

    bResult = M4_ERROR
    iRet = -1
    sFile = "C:\mydocuments\myexample.xlsm"

    'Load context
    iRet = SCH_JAVA_API.LOAD_CONTEXT("DATA_IMPORT")
    If iRet <> M4_SUCCESS Then
        Return(M4_ERROR)
    EndIf

    'Create factory
    oFactory =
    SCH_JAVA_API.CREATE_OBJECT("com.meta4.schemas.Plco_Es_Ws_Data_Imports
    erviceServiceLocator", iRet)
    If iRet <> M4_SUCCESS Then
        Return(M4_ERROR)
    EndIf

    oUrl = SCH_JAVA_API.CREATE_OBJECT("java.net.URL", iRet, "http://
    localhost:8100/services/PLCO_ES_WS_DATA_IMPORT")
    If iRet <> M4_SUCCESS Then
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        Return(M4_ERROR)
    EndIf

    'Create service
    oService = SCH_JAVA_API.INVOKE_METHOD(oFactory,
    "getPLCO_ES_WS_DATA_IMPORT", iRet, oUrl)
    If iRet <> M4_SUCCESS Then
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oUrl)
        Return(M4_ERROR)
    EndIf

    oBoolean = SCH_JAVA_API.CREATE_OBJECT("java.lang.Boolean", iRet,
    "true")
    If iRet <> M4_SUCCESS Then
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oUrl)

```

```

        Return(M4_ERROR)
    EndIf

    'Maintain session alive
    SCH_JAVA_API.INVOKE_METHOD(oService, "_setProperty", iRet,
    "javax.xml.rpc.session.maintain", oBoolean)

    'Destroy object
    SCH_JAVA_API.DESTROY_OBJECT(oBoolean)

    If iRet <> M4_SUCCESS Then
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    Return(M4_ERROR)
    EndIf

    'Retrieve session id
    SCH_JAVA_API.INVOKE_METHOD(oService, "retrieveM4Session", iRet,
    sSessionId)
    If iRet <> M4_SUCCESS Then
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oUrl)
        Return(M4_ERROR)
    EndIf

    bResult = M4_SUCCESS

    ' Load the file "C:\mydocuments\myexample.xlsm" into a DataHandler
    ' -----
    'Create FileDataSource object
    oFileDataSource =
    SCH_JAVA_API.CREATE_OBJECT("javax.activation.FileDataSource", iRet,
    sFile)
    If iRet <> M4_SUCCESS Then
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oUrl)
        Return(M4_ERROR)
    EndIf

    'Create DataHandler object
    oDataHandler =
    SCH_JAVA_API.CREATE_OBJECT("javax.activation.DataHandler", iRet,
    oFileDataSource)
    If iRet <> M4_SUCCESS Then
        SCH_JAVA_API.DESTROY_OBJECT(oFactory)
        SCH_JAVA_API.DESTROY_OBJECT(oUrl)
        SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
        Return(M4_ERROR)
    EndIf
    ' -----

```

```

' Build input objects.
' -----
'Create input block
oBlockInput =
SCH_JAVA_API.CREATE_OBJECT("com.meta4.schemas.Plco_Es_Ws_Data_ImportB
lock", iRet)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
    SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
    Return(M4_ERROR)
EndIf

oRecordInput =
SCH_JAVA_API.CREATE_OBJECT("com.meta4.schemas.Plco_Es_Ws_Data_ImportR
ecord", iRet)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
    SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
    SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
    Return(M4_ERROR)
EndIf

'Create empty array
oArrayList = SCH_JAVA_API.CREATE_OBJECT("java.util.ArrayList", iRet)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
    SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
    SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
    SCH_JAVA_API.DESTROY_OBJECT(oRecordInput)
    Return(M4_ERROR)
EndIf

'Assign empty record to array list (record set)
SCH_JAVA_API.INVOKE_METHOD(oArrayList, "add", iRet, oRecordInput)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
    SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
    SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
    SCH_JAVA_API.DESTROY_OBJECT(oRecordInput)
    SCH_JAVA_API.DESTROY_OBJECT(oArrayList)
    Return(M4_ERROR)
EndIf

```

```

'Assign record set to input block
SCH_JAVA_API.INVOKE_METHOD(oBlockInput,
"setPlco_Es_Ws_Data_ImportRecordSet", iRet, oArrayList)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
    SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
    SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
    SCH_JAVA_API.DESTROY_OBJECT(oRecordInput)
    SCH_JAVA_API.DESTROY_OBJECT(oArrayList)
    Return(M4_ERROR)
EndIf

'Assign file (data handler) to input block
SCH_JAVA_API.INVOKE_METHOD(oBlockInput, "setPlco_Data_File", iRet,
oDataHandler)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
    SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
    SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
    SCH_JAVA_API.DESTROY_OBJECT(oRecordInput)
    SCH_JAVA_API.DESTROY_OBJECT(oArrayList)
    Return(M4_ERROR)
EndIf

'Identify file name and assign to input block
sFileName = SCH_JAVA_API.INVOKE_METHOD(oDataHandler, "getName", iRet)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
    SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
    SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
    SCH_JAVA_API.DESTROY_OBJECT(oRecordInput)
    SCH_JAVA_API.DESTROY_OBJECT(oArrayList)
    Return(M4_ERROR)
EndIf

' -----

' Call the method.
' -----

'Invoke method to process file
oOutput = SCH_JAVA_API.INVOKE_METHOD(oService, "IMPORT_FILE", iRet,
"HF02", "PLCO_PY_PAYROLL_INTEG", sFileName, "myuser@meta4.com", 1,
oBlockInput)
If iRet <> M4_SUCCESS Then
    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)

```

```

SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
SCH_JAVA_API.DESTROY_OBJECT(oRecordInput)
SCH_JAVA_API.DESTROY_OBJECT(oArrayList)
Return(M4_ERROR)
EndIf

'Call successful, retrieve result
sResult = SCH_JAVA_API.INVOKE_METHOD(oOutput, "get_return", iRet)
' -----

' Process log messages.
' -----
oLogMessage = SCH_JAVA_API.INVOKE_METHOD(oOutput, "getLogMessage",
iRet)
If (iRet <> M4_SUCCESS Or IsNull(oLogMessage) = M4_TRUE )Then
SCH_JAVA_API.DESTROY_OBJECT(oFactory)
SCH_JAVA_API.DESTROY_OBJECT(oUrl)
SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
SCH_JAVA_API.DESTROY_OBJECT(oRecordInput)
SCH_JAVA_API.DESTROY_OBJECT(oArrayList)
Return(M4_ERROR)
EndIf

'Seems that object is not created if error queue is empty
iLogSize = SCH_JAVA_API.INVOKE_METHOD(oLogMessage, "size", iRet)
If iRet <> M4_SUCCESS Then
SCH_JAVA_API.DESTROY_OBJECT(oFactory)
SCH_JAVA_API.DESTROY_OBJECT(oUrl)
SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
SCH_JAVA_API.DESTROY_OBJECT(oRecordInput)
SCH_JAVA_API.DESTROY_OBJECT(oArrayList)
Return(M4_ERROR)
EndIf

'Push entire error queue of web service to current error queue
For i = 0 to iLogSize - 1
oLogMessagei = SCH_JAVA_API.INVOKE_METHOD(oLogMessage, "get",
iRet, i)
sCode = SCH_JAVA_API.INVOKE_METHOD(oLogMessagei, "getCode", iRet)
sSeverity = SCH_JAVA_API.INVOKE_METHOD(oLogMessagei,
"getSeverity", iRet)
sTitle = SCH_JAVA_API.INVOKE_METHOD(oLogMessagei, "getTitle",
iRet)
sDescription = SCH_JAVA_API.INVOKE_METHOD(oLogMessagei,
"getDescription", iRet)
iSeverity = M4_WARNINGLOG

```

```
        If sSeverity = "_error_" Then iSeverity = M4_ERRORLOG
        SetLog(iSeverity, 126, 3, 2, sTitle, sDescription)
    Next

    '-----
    '
    '-----
    '

    ologOutput = SCH_JAVA_API.INVOKE_METHOD(oServiceLogin, "logout",
    iRet)

    SCH_JAVA_API.DESTROY_OBJECT(oFactory)
    SCH_JAVA_API.DESTROY_OBJECT(oUrl)
    SCH_JAVA_API.DESTROY_OBJECT(oFileDataSource)
    SCH_JAVA_API.DESTROY_OBJECT(oDataHandler)
    SCH_JAVA_API.DESTROY_OBJECT(oBlockInput)
    SCH_JAVA_API.DESTROY_OBJECT(oRecordInput)
    SCH_JAVA_API.DESTROY_OBJECT(oArrayList)

    Return (M4_SUCCESS)
```
